



Université de Tunis El Manar – Faculté des Sciences de Tunis

Département Informatique

En partenariat avec **Tunisie Telecom**

Rapport de Projet de Fin d'Études

Pour l'obtention du diplôme de
Licence Nationale en Informatique

Sécurisation des flux de communication pour un écosystème IoT simulé

Architecture E2E – PKI – TLS/mTLS – SIEM – ML

Réalisé par :

Amine GHANNOUCHI

Encadrante universitaire : Mme. **ELLOUZE NOURHENE** – FST

Encadrant entreprise : M. Moez **KHLIFI** – Tunisie Telecom

Année universitaire **2025 – 2026**

Dédicace

*À mes parents, pour leur soutien indéfectible
et leur confiance tout au long de ce parcours.*

*À mes enseignants de la Faculté des Sciences de Tunis
qui m'ont transmis la passion de l'informatique.*

*À tous ceux qui œuvrent pour la sécurité
des systèmes d'information de demain.*

Remerciements

Ce travail n'aurait pu aboutir sans le concours de nombreuses personnes à qui je tiens à exprimer ma gratitude.

Je remercie tout d'abord **Mme. ELLOUZE NOURHENE**, encadrante universitaire à la Faculté des Sciences de Tunis, pour sa disponibilité, ses précieux conseils et son suivi rigoureux tout au long de ce projet.

Mes remerciements vont également à **M. Moez KHLIFI** de *Tunisie Telecom* pour m'avoir accueilli au sein de son équipe, orienté vers des problématiques réelles de l'industrie télécom et soutenu dans mes expérimentations techniques.

Je remercie l'ensemble du corps enseignant du **Département Informatique de la FST** pour la qualité de la formation dispensée.

Un grand merci aux équipes des projets open-source **Wazuh**, **HashiCorp Vault**, **Mosquitto** et **Suricata** dont les documentations exhaustives ont grandement facilité l'implémentation.

Enfin, je remercie ma famille et mes amis pour leur encouragement constant durant cette année académique.

Table des matières

Dédicace	i
Remerciements	iii
Liste des acronymes	xi
Introduction générale	1
1 Contexte général du projet	3
1.1 Cadre général du projet	3
1.2 Problématique	3
1.3 Objectifs spécifiques	4
1.4 Solution proposée (vue d'ensemble)	4
1.5 Présentation de l'organisme d'accueil	7
1.5.1 Tunisie Telecom	7
1.5.2 Service d'accueil	7
1.6 Conclusion	7
2 État de l'art	8
2.1 Introduction	8
2.2 Menaces et vulnérabilités spécifiques à l'IoT	8
2.2.1 Surface d'attaque étendue	8
2.2.2 OWASP IoT Top 10 et menaces réseau	8
2.2.3 Études de cas de référence	9
2.3 Protocoles applicatifs IoT	9
2.4 Sécurisation du transport : TLS et mTLS	9
2.4.1 TLS 1.3	9
2.4.2 Authentification mutuelle	10
2.5 Gestion des identités : PKI et automatisation	10
2.6 Détection d'intrusion et supervision	10
2.6.1 IDS réseau orienté MQTT	10
2.6.2 SIEM Wazuh	10
2.7 Apprentissage automatique pour la détection d'anomalies	11
2.8 Cas d'utilisation	11
2.9 Analyse critique de l'existant	13
2.10 Positionnement et synthèse	13
2.11 Conclusion	13

3	Conception	14
3.1	Introduction	14
3.2	Spécification des besoins	14
3.2.1	Besoins fonctionnels (BF)	14
3.2.2	Besoins non fonctionnels (BNF)	14
3.3	Choix méthodologique : étude comparative	15
3.4	Planification des sprints	16
3.5	Product Backlog	17
3.6	Conception UML	17
3.6.1	Diagramme de classes du simulateur	17
3.6.2	Diagramme de séquence : émission d'un certificat client	19
3.6.3	Diagramme d'activité : cycle de publication mTLS	19
3.7	Diagramme de Gantt	21
3.8	Conclusion	21
4	Réalisation	22
4.1	Introduction	22
4.2	Environnement de développement	22
4.3	Architecture globale et topologie réseau	22
4.4	Infrastructure à clés publiques (Vault)	27
4.4.1	Hiérarchie	27
4.4.2	Rôles Vault et émission	27
4.5	Broker MQTT en TLS 1.3 + mTLS	29
4.6	Scénarios d'attaque et contre-mesures	31
4.7	Pipeline d'apprentissage automatique	33
4.7.1	Données et features	33
4.7.2	Modèles	35
4.8	Captures d'écran des interfaces	35
4.9	Conclusion	36
5	Tests, résultats et conclusion	37
5.1	Introduction	37
5.2	Plan de tests	37
5.3	Résultats de sécurité	37
5.4	Résultats du module ML	38
5.4.1	Isolation Forest	38
5.4.2	Random Forest	39
5.5	Résultats de performance	41
5.5.1	Handshake TLS et latence	41
5.5.2	Débit et RTT	42

5.6	Limites identifiées	43
5.7	Perspectives	43
5.8	Conclusion générale	44
Références bibliographiques		45
A Scripts et Code Source		47
A.1	Bootstrap de la PKI — Vault API	47
A.2	Émission du certificat Mosquitto	50
A.3	Génération des certificats devices	50
A.4	Génération du fichier ACL Mosquitto	52
A.5	Tests de connectivité réseau	52
A.6	Tests de performance MQTT	53
A.7	Simulateur de flotte IoT	55
A.8	Analyse des événements Suricata	56
B Fichiers de Configuration		58
B.1	Configuration MikroTik CHR	58
B.2	Configuration Mosquitto (TLS/mTLS)	60
B.3	Configuration Suricata (MQTT natif)	61
B.4	Règles Suricata personnalisées	62
B.5	Règles Wazuh personnalisées	63
B.6	Dockerfile du simulateur de flotte	64
B.7	Dockerfile Toolbox	65
B.8	Commandes de création des réseaux Docker	65
B.9	Variables d’environnement du projet	66

Table des figures

1.1	Vue d'ensemble de l'architecture de sécurisation proposée	6
2.1	Diagramme de cas d'utilisation de la plateforme	12
3.1	Enchaînement des six sprints	16
3.2	Diagramme de classes du simulateur de flotte	18
3.3	Séquence d'émission d'un certificat client	19
3.4	Activité d'un dispositif : chargement des secrets, handshake, publication	20
3.5	Diagramme de Gantt du projet	21
4.1	Architecture globale en défense en profondeur	23
4.2	Topologie réseau détaillée (VLANs et plan d'adressage)	24
4.3	Flux E2E — Phase 1 : provisionnement de l'identité	24
4.4	Flux E2E — Phase 2 : établissement de la connexion TLS 1.3 mTLS .	25
4.5	Flux E2E — Phase 3 : publication des données IoT	25
4.6	Flux E2E — Phase 4 : détection et supervision	26
4.7	Flux E2E — Phase 5 : rotation automatique des certificats	26
4.8	Hierarchie de la PKI	28
4.9	Handshake TLS 1.3 avec authentification mutuelle	30
4.10	Chaîne de détection Suricata → Wazuh	32
4.11	Pipeline d'apprentissage automatique	33
4.12	Analyse exploratoire — distribution des classes	34
4.13	Analyse exploratoire — matrice de corrélation des features numériques	34
4.14	Vault UI – vue de la hiérarchie PKI	35
4.15	Wazuh Dashboard – alertes corrélées	35
4.16	GNS3 – topologie déployée du laboratoire	36
5.1	Isolation Forest – matrice de confusion	38
5.2	Isolation Forest – distribution des scores d'anomalie	39
5.3	Random Forest : matrice de confusion et courbe ROC	39
5.4	Random Forest – confusion multi-classes (8 classes)	40
5.5	Random Forest – importance des features	40
5.6	Comparaison synthétique des modèles	41
5.7	Distribution du handshake mTLS	41
5.8	Latence de connexion	42
5.9	Débit applicatif	42
5.10	RTT par taille de message	42

5.11 Comparaison MQTT clair vs MQTT/TLS vs MQTT/mTLS	43
--	----

Liste des tableaux

1.1	Objectifs spécifiques du projet	4
2.1	Comparaison des principaux protocoles applicatifs IoT	9
3.1	Besoins fonctionnels	14
3.2	Besoins non fonctionnels	15
3.3	Comparaison Scrum vs Kanban vs XP	15
3.4	Planification des sprints	16
3.5	Product Backlog (extrait priorisé)	17
4.1	Composants de l'environnement de développement	22
5.1	Plan de tests	37
5.2	Taux de détection des scénarios d'attaque (10 rejeux par scénario) . . .	38
5.3	Performances ML (validation 5-fold)	41
B.1	Variables d'environnement principales du projet	66

Liste des acronymes

IoT	Internet of Things – Internet des Objets
TLS	Transport Layer Security
mTLS	Mutual TLS – TLS mutuel (authentification bidirectionnelle)
DTLS	Datagram TLS
PKI	Public Key Infrastructure – Infrastructure à Clés Publiques
CA	Certificate Authority – Autorité de Certification
CSR	Certificate Signing Request
CRL	Certificate Revocation List
OCSP	Online Certificate Status Protocol
MQTT	Message Queuing Telemetry Transport
CoAP	Constrained Application Protocol
HTTP	HyperText Transfer Protocol
AMQP	Advanced Message Queuing Protocol
SIEM	Security Information and Event Management
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
DPI	Deep Packet Inspection
DoS	Denial of Service
DDoS	Distributed Denial of Service
MiTM	Man-in-the-Middle
ML	Machine Learning – Apprentissage Automatique
RF	Random Forest
IF	Isolation Forest

ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
RTT	Round-Trip Time
QoS	Quality of Service
DMZ	DeMilitarized Zone
VLAN	Virtual Local Area Network
GNS3	Graphical Network Simulator 3
VM	Virtual Machine
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
ACL	Access Control List
CN	Common Name
SAN	Subject Alternative Name
E2E	End-to-End
RSA	Rivest–Shamir–Adleman
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
OVA	Open Virtual Appliance
TT	Tunisie Telecom
FST	Faculté des Sciences de Tunis
OWASP	Open Worldwide Application Security Project
ENISA	European Union Agency for Cybersecurity
NIST	National Institute of Standards and Technology
RFC	Request For Comments
ADSL	Asymmetric Digital Subscriber Line

LPWAN Low-Power Wide-Area Network

SOC Security Operations Center

ETL Extract Transform Load

BF-1 Besoin Fonctionnel n°1

BF-2 Besoin Fonctionnel n°2

BF-3 Besoin Fonctionnel n°3

BF-4 Besoin Fonctionnel n°4

BF-5 Besoin Fonctionnel n°5

BF-6 Besoin Fonctionnel n°6

BNF-1 Besoin Non Fonctionnel n°1

BNF-2 Besoin Non Fonctionnel n°2

BNF-3 Besoin Non Fonctionnel n°3

BNF-4 Besoin Non Fonctionnel n°4

BNF-5 Besoin Non Fonctionnel n°5

BNF-6 Besoin Non Fonctionnel n°6

O1 Objectif n°1

O2 Objectif n°2

O3 Objectif n°3

O4 Objectif n°4

O5 Objectif n°5

O6 Objectif n°6

O7 Objectif n°7

Introduction générale

1. L’Internet des Objets et la sécurité des communications

L’**Internet des Objets** (IoT, *Internet of Things*) désigne l’interconnexion de dispositifs physiques capables de collecter, transmettre et exploiter des données dans des environnements très variés : villes intelligentes, santé connectée, industrie, agriculture, transport ou infrastructures de télécommunications [1]. Ces dispositifs peuvent être de simples capteurs, des passerelles industrielles, des compteurs intelligents ou des équipements embarqués. Leur rôle est de rapprocher le monde physique des systèmes numériques afin d’automatiser la surveillance, l’analyse et la prise de décision.

Cette évolution apporte cependant de nouveaux risques. Contrairement aux systèmes informatiques classiques, les objets connectés sont souvent nombreux, hétérogènes, parfois peu puissants et rarement administrés manuellement après leur déploiement. Ils communiquent sur des réseaux partagés, publient des données sensibles et s’appuient sur des protocoles applicatifs légers comme MQTT ou CoAP. La moindre faiblesse d’authentification, de chiffrement ou de segmentation peut donc exposer l’ensemble de l’écosystème à l’interception, à l’usurpation d’identité, au déni de service ou à l’exfiltration de données.

La sécurité des communications IoT doit ainsi être pensée de bout en bout. Elle ne se limite pas à chiffrer un canal réseau : elle implique la gestion fiable des identités, l’émission et la rotation de certificats, l’authentification mutuelle entre dispositifs et services, la restriction des accès par politiques, ainsi qu’une supervision capable de détecter les comportements anormaux. Dans un contexte opérateur, ces exigences deviennent encore plus importantes, car l’infrastructure peut héberger plusieurs services, plusieurs clients et des volumes élevés de messages.

Ce projet de fin d’études s’inscrit dans cette problématique. Il vise à concevoir et valider, dans un environnement simulé, une architecture de sécurisation des flux IoT combinant PKI, TLS/mTLS, broker MQTT sécurisé, segmentation réseau, détection IDS/SIEM et apprentissage automatique. L’objectif est de proposer une chaîne reproductible, mesurable et adaptée aux contraintes d’un écosystème IoT représentatif.

2. Structure du rapport

Le rapport est organisé en cinq chapitres. Le **chapitre 1** présente le contexte général du projet, l’organisme d’accueil, la problématique, les objectifs et la solution proposée.

Le **chapitre 2** expose l'état de l'art relatif aux menaces IoT, aux protocoles, aux mécanismes de sécurité et aux outils de supervision. Le **chapitre 3** détaille la conception : besoins, choix méthodologique, planification Scrum, backlog et diagrammes UML. Le **chapitre 4** décrit la réalisation technique de l'architecture, les scénarios d'attaque, le pipeline ML et les captures d'interfaces. Enfin, le **chapitre 5** présente les tests, les résultats, les limites, les perspectives et la conclusion générale.

Chapitre 1

Contexte général du projet

1.1 Cadre général du projet

Ce premier chapitre précise le cadre dans lequel s'inscrit le projet. Il présente l'environnement d'accueil, le besoin opérationnel identifié, la problématique de sécurité traitée, les objectifs techniques retenus et la solution proposée. L'introduction générale ayant déjà posé les enjeux globaux de l'IoT, ce chapitre se concentre sur le contexte concret du PFE et sur les choix qui orientent la suite du rapport.

Dans le cadre de ce travail, l'objectif est de construire un prototype expérimental capable de sécuriser les flux de communication d'un écosystème IoT simulé. Le projet s'appuie sur une architecture représentative d'un environnement opérateur : segmentation réseau, services de confiance, broker MQTT, supervision de sécurité et analyse comportementale. Cette approche permet de valider les mécanismes proposés avant toute projection vers un déploiement réel à plus grande échelle.

1.2 Problématique

L'écosystème d'un opérateur télécom présente plusieurs spécificités qui rendent la sécurisation des flux IoT particulièrement délicate :

- **Hétérogénéité** des dispositifs (microcontrôleurs ESP32, passerelles industrielles, modems cellulaires) et des protocoles applicatifs (MQTT, CoAP, AMQP, LwM2M) ;
- **Volumétrie** massive et asynchrone des messages (publications périodiques, événements rares mais critiques) ;
- **Contraintes embarquées** : ressources CPU/mémoire limitées, batteries, latence radio variable ;
- **Multi-tenance** et segmentation réseau : un même opérateur héberge plusieurs clients verticaux qui ne doivent jamais accéder aux flux des autres ;
- **Gestion à grande échelle** d'identités cryptographiques (certificats X.509) pour des dizaines de milliers d'objets, avec des cycles de vie variables.

La problématique centrale de ce projet de fin d'études peut être formulée ainsi :

Problématique

*Comment concevoir, déployer et valider, dans un environnement simulé représentatif d'un opérateur télécom, une architecture de sécurisation **de bout en bout** des flux de communication d'un écosystème IoT, qui combine : (i) un chiffrement et une authentification mutuelle systématiques entre objets et infrastructures ; (ii) une gestion automatisée du cycle de vie des certificats ; (iii) une supervision active des menaces par corrélation d'alertes réseau et apprentissage automatique ; (iv) tout en restant compatible avec les contraintes de performance et de scalabilité du domaine ?*

1.3 Objectifs spécifiques

À partir de cette problématique, sept objectifs spécifiques (O1–O7) structurent l'ensemble des travaux :

Tab. 1.1 : Objectifs spécifiques du projet

ID	Objectif
O1	Concevoir une architecture IoT segmentée et reproductible, déployable dans un environnement simulé.
O2	Déployer une infrastructure à clés publiques (PKI) automatisée fondée sur HashiCorp Vault.
O3	Imposer un chiffrement TLS 1.3 avec authentification mutuelle (mTLS) sur le broker MQTT.
O4	Simuler une flotte hétérogène de 50 dispositifs IoT et générer un trafic réaliste mêlant comportements légitimes et attaques.
O5	Mettre en place une analyse réseau native MQTT au moyen d'un IDS Suricata.
O6	Centraliser et corréler les événements de sécurité dans un SIEM Wazuh.
O7	Développer un module de détection d'anomalies par apprentissage automatique (IsolationForest et Random Forest) et évaluer son apport.

1.4 Solution proposée (vue d'ensemble)

Pour répondre à la problématique et atteindre les objectifs ci-dessus, nous proposons une **architecture de défense en profondeur** composée de six briques complémentaires, intégralement déployée dans un laboratoire simulé sous GNS3 [2] et Docker [3] :

1. Une **infrastructure réseau segmentée** (VLANs IoT, services, monitoring, management) construite autour d'un pare-feu pfSense et d'un routeur MikroTik ;

2. Une **PKI automatisée** HashiCorp Vault à deux niveaux (CA racine hors ligne + CA intermédiaire en ligne), capable d'émettre à la demande des certificats X.509 pour les services et les objets [4] ;
3. Un **broker MQTT Mosquitto** configuré en TLS 1.3 avec authentification mutuelle obligatoire et listes de contrôle d'accès par topic [5] ;
4. Un **simulateur de flotte** de 50 dispositifs Python jouant 76 000 messages issus d'un jeu de données réel, dont sept catégories d'attaques ;
5. Une **chaîne de détection** comprenant un IDS Suricata avec règles MQTT personnalisées [6] et un SIEM Wazuh [7] pour la corrélation et la visualisation ;
6. Un **pipeline de ML** (IsolationForest [8] et Random Forest [9]) entraîné sur les caractéristiques extraites du trafic, capable de détecter les anomalies non couvertes par les règles statiques.

La figure 1.1 schématise cette vision globale, qui sera détaillée et justifiée dans les chapitres suivants.

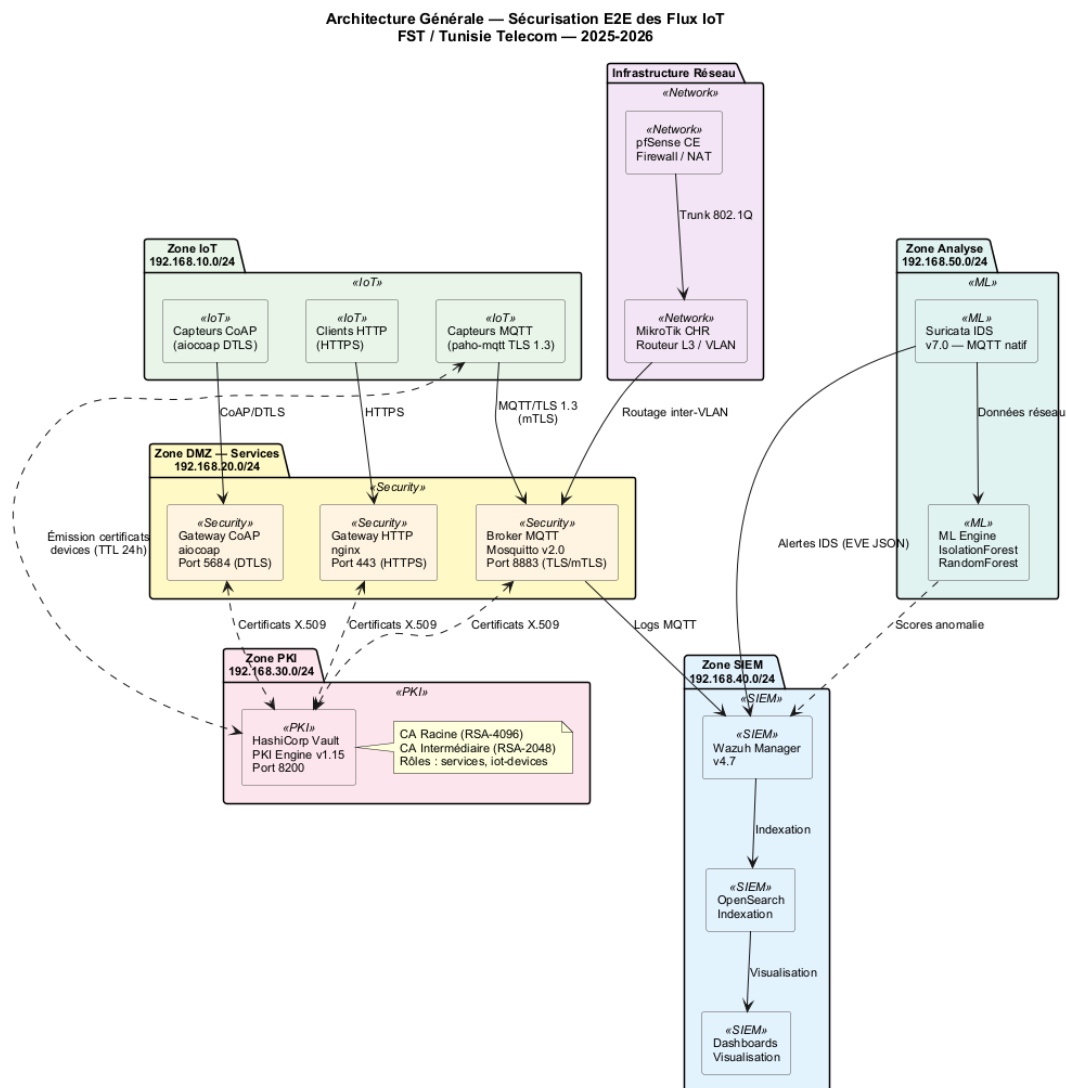


Fig. 1.1 : Vue d'ensemble de l'architecture de sécurisation proposée

1.5 Présentation de l'organisme d'accueil

1.5.1 Tunisie Telecom

Tunisie Telecom (Société Nationale des Télécommunications, *Itissalat Tounes*) est l'opérateur historique de télécommunications de la Tunisie, fondé en 1995 [10]. Présent à l'échelle nationale, l'opérateur fournit des services de téléphonie fixe et mobile, d'accès Internet (ADSL, fibre, 4G/5G), d'hébergement, de cloud et de solutions entreprises. Avec un parc client de plusieurs millions d'abonnés et une infrastructure couvrant l'ensemble du territoire, *Tunisie Telecom* occupe une position centrale dans la transformation numérique du pays.

Dans le domaine de l'IoT, *Tunisie Telecom* accompagne depuis plusieurs années des clients industriels et des acteurs publics dans le déploiement de solutions verticales (smart metering, suivi de flotte, surveillance de sites distants, agriculture connectée). Cette dynamique impose à l'opérateur de maîtriser non seulement la *connectivité* (réseaux LPWAN, cellulaire), mais également les *couches applicatives* et les *exigences de sécurité* associées.

1.5.2 Service d'accueil

Le présent projet a été réalisé au sein du service en charge des *infrastructures de cybersécurité et des projets d'innovation*. Ce service intervient sur trois axes complémentaires : (i) la **veille technologique** sur les nouvelles menaces et les contre-mesures émergentes ; (ii) le **prototypage** en environnement de laboratoire d'architectures candidates avant industrialisation ; (iii) la **validation expérimentale** de ces architectures par campagnes de tests fonctionnels, de sécurité et de performance.

C'est dans ce cadre que s'inscrit notre travail : concevoir un *prototype de référence* de sécurisation des flux IoT, entièrement reproductible en environnement simulé, et utilisable comme socle pour les futures industrialisations.

1.6 Conclusion

Ce chapitre a défini le contexte général du projet, son ancrage au sein de *Tunisie Telecom*, la problématique de sécurisation des flux IoT, les objectifs spécifiques et la solution proposée. Le chapitre suivant complète ce cadrage par l'état de l'art des menaces, protocoles, standards et outils mobilisés dans cette architecture.

Chapitre 2

État de l’art

2.1 Introduction

Avant de concevoir une architecture de sécurisation des flux IoT, il est indispensable de cadrer le sujet sur trois plans complémentaires : (i) caractériser les **menaces** qui pèsent spécifiquement sur les écosystèmes IoT à grande échelle ; (ii) recenser les **technologies et protocoles** candidats pour le transport, l’authentification et la supervision ; (iii) analyser les **travaux et solutions existants** afin d’en extraire les bonnes pratiques et de positionner notre contribution. Ce chapitre est structuré autour de ces trois axes, suivi de la description des **cas d’utilisation** retenus, d’une analyse critique et d’une synthèse positionnant le projet.

2.2 Menaces et vulnérabilités spécifiques à l’IoT

2.2.1 Surface d’attaque étendue

Les écosystèmes IoT présentent une surface d’attaque significativement plus large que les systèmes informatiques classiques. Meneghello *et al.* [1] en distinguent quatre couches : (i) la couche *matérielle* (composants embarqués, JTAG/UART exposés, attaques par canal auxiliaire) ; (ii) la couche *firmware* (mises à jour non signées, exécution de code arbitraire) ; (iii) la couche *réseau* (transports en clair, protocoles propriétaires faibles) ; (iv) la couche *services et écosystème* (API non authentifiées, applications mobiles vulnérables, multi-tenance mal cloisonnée).

2.2.2 OWASP IoT Top 10 et menaces réseau

L’OWASP synthétise dans son *IoT Top 10* [11] les dix risques applicatifs majeurs : mots de passe faibles ou par défaut, services réseau non sécurisés, interfaces écosystème non protégées, mécanisme de mise à jour absent, composants obsolètes, protection insuffisante de la vie privée, transferts et stockage non chiffrés, absence de gestion des dispositifs, paramètres par défaut non sûrs, absence de hardening physique. Dans ce projet, nous nous concentrons sur les menaces **réseau** et **applicatives** : interception (*sniffing*), *man-in-the-middle*, force brute, exfiltration applicative, déni de service, vol/abus de certificats.

2.2.3 Études de cas de référence

L’analyse de l’attaque **Mirai** [12], [13] illustre toutes ces faiblesses simultanément : le botnet exploitait des couples identifiant/mot de passe par défaut sur des objets exposés en SSH/Telnet, recrutait des centaines de milliers de dispositifs en quelques jours, et générait des attaques DDoS dépassant 1 Tbit/s. Le rapport *ENISA Threat Landscape for IoT* [14] confirme que ces vecteurs restent dominants et identifie comme contre-mesures prioritaires : l’authentification forte, le chiffrement systématique, la segmentation réseau et la supervision active. Ces orientations rejoignent les recommandations du NIST [15] pour la cybersécurité des dispositifs IoT.

2.3 Protocoles applicatifs IoT

Le tableau 2.1 compare quatre protocoles applicatifs représentatifs.

Tab. 2.1 : Comparaison des principaux protocoles applicatifs IoT

Critère	MQTT 5	CoAP	AMQP 1.0	HTTP/REST
Modèle	Pub/Sub	Req/Rep	Pub/Sub queues	+ Req/Rep
Transport	TCP/TLS	UDP/DTLS	TCP/TLS	TCP/TLS
Empreinte	Très faible	Très faible	Moyenne	Moyenne/élevée
QoS	0/1/2	Confirmable	0/1	Aucun natif
Sécurité native	TLS+ACL	DTLS+OSCORE	SASL+TLS	TLS+JWT
Maturité IoT	Très large	Large	Industrielle	Universelle

Le choix de **MQTT 5** [16] pour ce projet repose sur trois critères : empreinte mémoire et réseau adaptée aux microcontrôleurs ; large adoption dans les écosystèmes industriels et les opérateurs IoT ; existence de mécanismes natifs d’authentification (TLS+ACL) facilement combinables avec mTLS.

2.4 Sécurisation du transport : TLS et mTLS

2.4.1 TLS 1.3

Le protocole TLS 1.3 [17] apporte par rapport à TLS 1.2 une simplification significative : suppression des suites cryptographiques obsolètes, négociation en **1-RTT** (et 0-RTT optionnel), *forward secrecy* obligatoire, signatures et échanges de clés modernisés (ECDHE, EdDSA, AES-GCM, ChaCha20-Poly1305). Les recommandations opérationnelles publiées dans la RFC 9325 [18] guident le choix des paramètres et la durée de vie des certificats.

2.4.2 Authentification mutuelle

Dans un déploiement classique, seul le serveur s'authentifie auprès du client. Le **mTLS** (*mutual TLS*) impose en outre au client de présenter un certificat X.509 valide signé par une autorité de confiance. Cette double authentification permet : (i) d'éviter qu'un objet illégitime se connecte au broker ; (ii) de chiffrer chaque session par bout ; (iii) de tracer chaque message via l'identité cryptographique de son émetteur. Cette propriété est essentielle pour la mise en place ultérieure d'ACL par certificat dans Mosquitto [5].

2.5 Gestion des identités : PKI et automatisations

Une PKI à deux niveaux (CA racine *offline* + CA intermédiaire *online*) constitue la pratique recommandée [15] : la CA racine ne signe que des CA intermédiaires, et reste hors ligne ; les CA intermédiaires signent les certificats opérationnels (services, dispositifs). En cas de compromission d'une CA intermédiaire, seul le sous-arbre concerné doit être révoqué.

L'automatisation de l'émission est indispensable lorsque le parc dépasse quelques dizaines d'objets. **HashiCorp Vault** [4] fournit un *secrets engine pki* qui expose une API HTTP de demande de certificats avec contrôle fin (rôles, ACL, TTL). C'est la solution retenue dans ce projet.

2.6 Détection d'intrusion et supervision

2.6.1 IDS réseau orienté MQTT

Suricata est un IDS/IPS open source qui dispose, depuis la version 6, d'un *parser* natif MQTT capable d'extraire les champs **CONNECT**, **PUBLISH**, **SUBSCRIBE** [6]. Cette capacité permet de définir des règles métier (par exemple : alerter sur un **CONNECT** sans **ClientID** reconnu, ou sur une rafale de **CONNECT** indiquant un *brute force*) qui ne seraient pas accessibles à un IDS générique.

2.6.2 SIEM Wazuh

Wazuh [7] est une plateforme de *Security Information and Event Management* qui agrège les journaux d'agents endpoint, de sources réseau (Suricata) et applicatifs (Vault, Mosquitto). Elle s'intègre nativement à OpenSearch pour le stockage et au tableau de bord Wazuh Dashboard pour la visualisation et la corrélation. Sa modularité (règles XML, décodeurs personnalisables) en fait un candidat naturel pour la consolidation des alertes IoT.

2.7 Apprentissage automatique pour la détection d’anomalies

Les approches par règles statiques sont efficaces pour les attaques connues mais peinent à détecter les comportements nouveaux. Hasan *et al.* [19] ont montré qu’un classifieur supervisé entraîné sur des features réseau et applicatives obtient des taux de détection supérieurs à 99 % sur sept catégories d’attaques IoT. Les algorithmes les plus utilisés sont :

- **Isolation Forest** [8] : méthode non supervisée d’isolement, particulièrement efficace en haute dimension et adaptée aux scénarios où les anomalies sont rares ;
- **Random Forest** [9] : ensemble d’arbres de décision, robuste, interprétable (*feature importance*) et performant en classification multi-classes.

La bibliothèque *scikit-learn* [20] fournit des implémentations matures de ces deux algorithmes ; elle constitue le socle de notre pipeline ML.

2.8 Cas d’utilisation

Trois acteurs principaux interagissent avec la plateforme proposée : le **dispositif IoT** (publie des télémessures), l’**administrateur** (configure et supervise) et l’**analyste sécurité** (consulte les alertes et investigate). La figure 2.1 synthétise leurs cas d’utilisation.

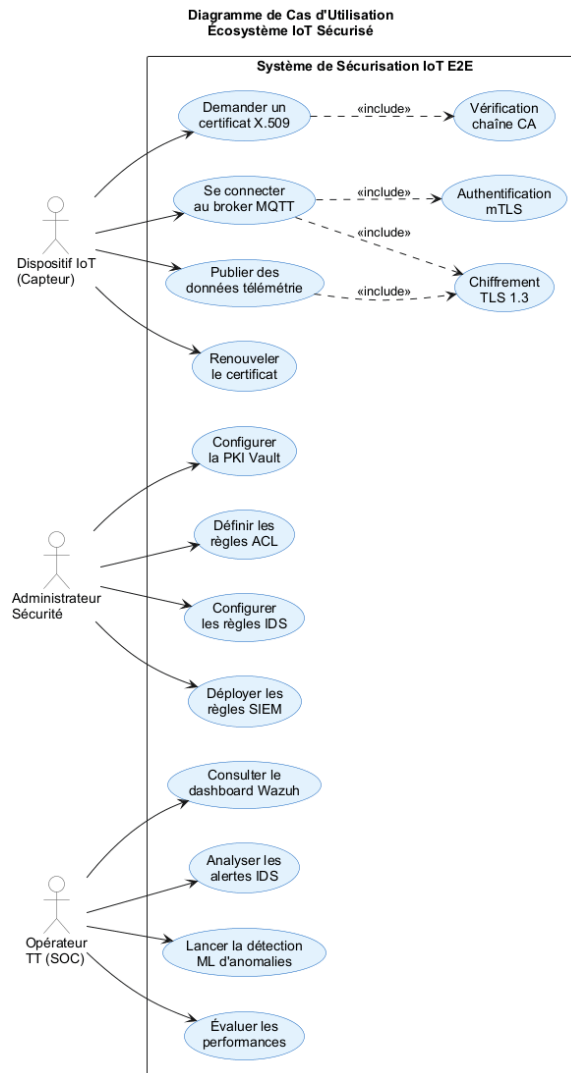


Fig. 2.1 : Diagramme de cas d'utilisation de la plateforme

2.9 Analyse critique de l’existant

Plusieurs approches ont été proposées pour sécuriser les flux IoT : gateways IoT propriétaires (AWS IoT Core, Azure IoT Hub), distributions intégrées (ThingsBoard, Eclipse Kura), ou architectures sur mesure. Notre analyse fait ressortir quatre limitations récurrentes :

1. **Verrouillage fournisseur** pour les solutions cloud commerciales, peu compatible avec une stratégie souveraine d’opérateur télécom ;
2. **Couverture partielle** de la sécurité chez les distributions intégrées (souvent TLS oui, mais pas mTLS systématique ni PKI automatisée) ;
3. **Faible intégration** entre la couche réseau (IDS) et la couche applicative (broker, ML) ;
4. **Reproductibilité limitée** des architectures publiées dans la littérature, rarement accompagnées d’un environnement de simulation complet.

2.10 Positionnement et synthèse

Au regard de cet état de l’art, le présent projet se positionne comme une **architecture de référence open source** pour la sécurisation des flux IoT chez un opérateur télécom, avec quatre contributions distinctives : (1) déploiement de bout en bout reproductible sous GNS3+Docker ; (2) PKI Vault entièrement automatisée avec mTLS systématique ; (3) chaîne de détection unifiée Suricata + Wazuh avec règles MQTT spécifiques ; (4) couplage de cette détection signature avec un module ML (IsolationForest + RandomForest) pour couvrir les anomalies non connues.

2.11 Conclusion

Ce chapitre a établi le socle conceptuel du projet : panorama des menaces, comparaison des protocoles, principes de TLS/mTLS, mécanismes PKI, outils de détection et apport de l’apprentissage automatique. Les cas d’utilisation et le positionnement synthétisent les contributions visées. Le chapitre suivant traduit ces orientations en **conception détaillée** : besoins, méthodologie, planification, backlog et modélisation UML.

Chapitre 3

Conception

3.1 Introduction

Ce chapitre traduit les besoins exprimés au chapitre précédent en une conception structurée. Nous formalisons d’abord les besoins fonctionnels et non fonctionnels. Nous comparons ensuite trois méthodologies agiles candidates (Scrum, Kanban, XP) et justifions le choix de Scrum. La planification des sprints précède la présentation du *Product Backlog* afin de garantir une lecture chronologique cohérente. Enfin, nous présentons les diagrammes UML de conception et le diagramme de Gantt récapitulatif.

3.2 Spécification des besoins

3.2.1 Besoins fonctionnels (BF)

Tab. 3.1 : Besoins fonctionnels

ID	Description
BF-1	Émettre, renouveler et révoquer automatiquement des certificats X.509 pour services et dispositifs.
BF-2	Imposer le chiffrement TLS 1.3 et l’authentification mutuelle (mTLS) sur le broker MQTT.
BF-3	Restreindre l’accès aux topics MQTT par dispositif via des listes de contrôle d’accès.
BF-4	Simuler une flotte hétérogène de 50 dispositifs IoT générant un trafic réaliste.
BF-5	Détecter les attaques réseau et applicatives par règles IDS et corréler les alertes dans un SIEM.
BF-6	Détecter les anomalies non couvertes par les règles via un modèle d’apprentissage automatique.

3.2.2 Besoins non fonctionnels (BNF)

Tab. 3.2 : Besoins non fonctionnels

ID	Description
BNF-1	Sécurité : aucun flux IoT en clair ; rotation des secrets ; séparation CA racine / intermédiaire.
BNF-2	Performance : latence handshake mTLS < 50 ms ; débit broker > 500 msg/s par client.
BNF-3	Reproductibilité : l’environnement complet doit être réinstallable à partir des scripts fournis.
BNF-4	Scalabilité : l’architecture doit accepter sans refonte au moins 200 dispositifs.
BNF-5	Maintenabilité : configurations versionnées, modules indépendants, journalisation centralisée.
BNF-6	Observabilité : tableau de bord Wazuh accessible aux analystes en moins de 3 clics.

3.3 Choix méthodologique : étude comparative

Trois méthodologies agiles candidates ont été évaluées : **Scrum** [21], **Kanban** [22] et **Extreme Programming (XP)** [23]. Le tableau 3.3 résume la comparaison sur sept critères.

Tab. 3.3 : Comparaison Scrum vs Kanban vs XP

Critère	Scrum	Kanban	XP
Cadre de travail	Itératif (sprints)	Continu (flux)	Itératif court
Taille équipe	3–9 personnes	Variable	2–12
Gestion des changements	Encadrée (entre sprints)	Très flexible	Flexible
Cycle de livraison	1–4 semaines	Continu	1–3 semaines
Documentation	Modérée (US, backlog)	Faible	Faible
Pratiques d’ingénierie	Indépendantes	Indépendantes	Strictes (TDD, pair-prog)
Courbe d’apprentissage	Modérée	Faible	Élevée
Adaptation au PFE	Très adaptée	Peu adaptée (jalons académiques)	Peu adaptée (équipe solo)

Justification du choix Scrum. Le PFE impose des jalons académiques fixes (revues d’avancement, rapport intermédiaire, soutenance), ce qui correspond naturellement à la cadence des sprints. La taille modeste de l’équipe (un étudiant + un encadrant

universitaire + un encadrant industriel) reste compatible avec les rituels Scrum. Enfin, Scrum offre un compromis entre la rigueur (revues systématiques) et la flexibilité (réajustement du backlog entre sprints), ce qui le rend supérieur à Kanban (trop ouvert pour un projet calendaire) et à XP (pratiques d’ingénierie disproportionnées pour un solo).

3.4 Planification des sprints

Six sprints de deux semaines structurent la phase de réalisation. Le tableau 3.4 en présente les objectifs et les livrables. Cette planification précède intentionnellement le *Product Backlog* : elle fournit la grille de lecture qui permet ensuite d’affecter les *User Stories* aux sprints concernés.

Tab. 3.4 : Planification des sprints

Sprint	Durée	Objectif principal
S1 – Cadrage & réseau	2 semaines	Topologie GNS3 segmentée, conteneurs de base, plan d’adressage.
S2 – PKI Vault	2 semaines	Vault déployé, CA racine + intermédiaire, rôles PKI, scripts d’émission.
S3 – Broker mTLS	2 semaines	Mosquitto en TLS 1.3+mTLS, ACL par certificat, tests d’authentification.
S4 – Flotte simulée	2 semaines	50 clients Python, génération de trafic, rejeu de 76 000 messages.
S5 – Détection IDS+SIEM	2 semaines	Règles Suricata MQTT, intégration Wazuh, dashboards.
S6 – ML & bilan	2 semaines	Modèles IsolationForest+RandomForest, évaluation, rédaction finale.

La figure 3.1 illustre l’enchaînement des sprints et leurs dépendances internes.

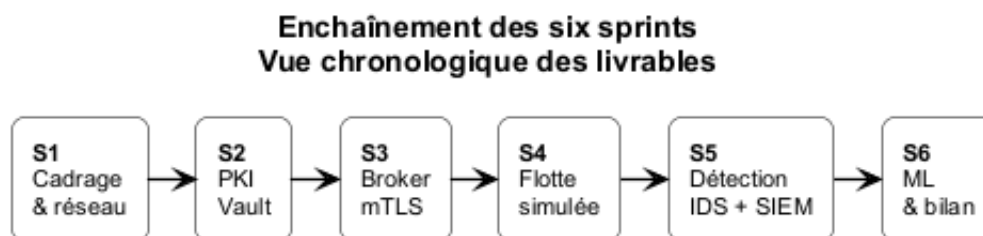


Fig. 3.1 : Enchaînement des six sprints

3.5 Product Backlog

Le *Product Backlog* regroupe les *User Stories* priorisées selon la méthode **MoSCoW** (*Must, Should, Could, Won't*) et estimées en *story points* (échelle de Fibonacci modifiée). Chaque US est rattachée à un sprint cible.

Tab. 3.5 : Product Backlog (extrait priorisé)

ID	Acteur	User Story	Prio.	SP	Sprint
US-01	Admin	Déployer une topologie réseau segmentée sous GNS3	M	8	S1
US-02	Admin	Définir un plan d'adressage par VLAN	M	3	S1
US-03	Admin	Démarrer Vault et initialiser une CA racine	M	5	S2
US-04	Admin	Provisionner une CA intermédiaire et des rôles PKI	M	8	S2
US-05	Admin	Émettre des certificats serveur/client à la demande	M	5	S2
US-06	Admin	Configurer Mosquitto en TLS 1.3 + mTLS	M	8	S3
US-07	Admin	Définir des ACL par identifiant client	M	5	S3
US-08	Dispositif	Établir une session mTLS et publier sur son topic	M	5	S3
US-09	Admin	Simuler 50 dispositifs hétérogènes	M	8	S4
US-10	Admin	Rejouer un dataset de 76 000 messages	S	5	S4
US-11	Analyste	Détecter une connexion non autorisée	M	5	S5
US-12	Analyste	Détecter un brute-force d'authentification	M	5	S5
US-13	Analyste	Visualiser les alertes corrélées dans Wazuh	M	5	S5
US-14	Analyste	Détecter une anomalie via IsolationForest	S	8	S6
US-15	Analyste	Classifier les attaques via Random Forest	S	8	S6
US-16	Analyste	Comparer ML vs règles IDS	C	5	S6

Soit un total estimé à 96 story points sur 6 sprints (~ 16 SP/sprint), conforme à la capacité d'une équipe solo avec encadrement.

3.6 Conception UML

3.6.1 Diagramme de classes du simulateur

Le simulateur de flotte est structuré autour de quatre classes principales : **DeviceProfile** (caractéristiques d'un type de dispositif), **IoTDevice** (instance simulée), **TrafficGenerator** (orchestration du jeu) et **AttackInjector** (injection des sept catégories d'attaques).

La figure 3.2 en donne la vue.

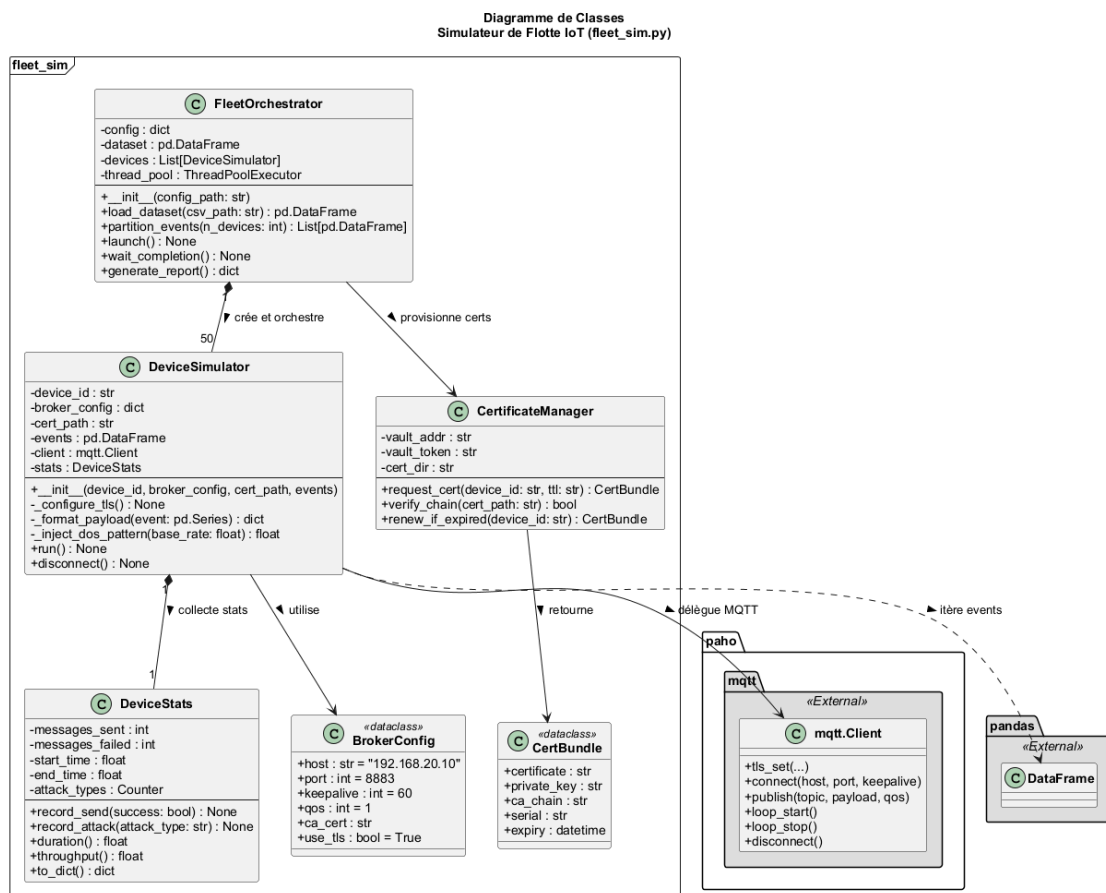


Fig. 3.2 : Diagramme de classes du simulateur de flotte

3.6.2 Diagramme de séquence : émission d'un certificat client

L'émission d'un certificat suit un échange en cinq étapes entre le dispositif (ou le script de provisionnement), Vault et la CA intermédiaire (figure 3.3).

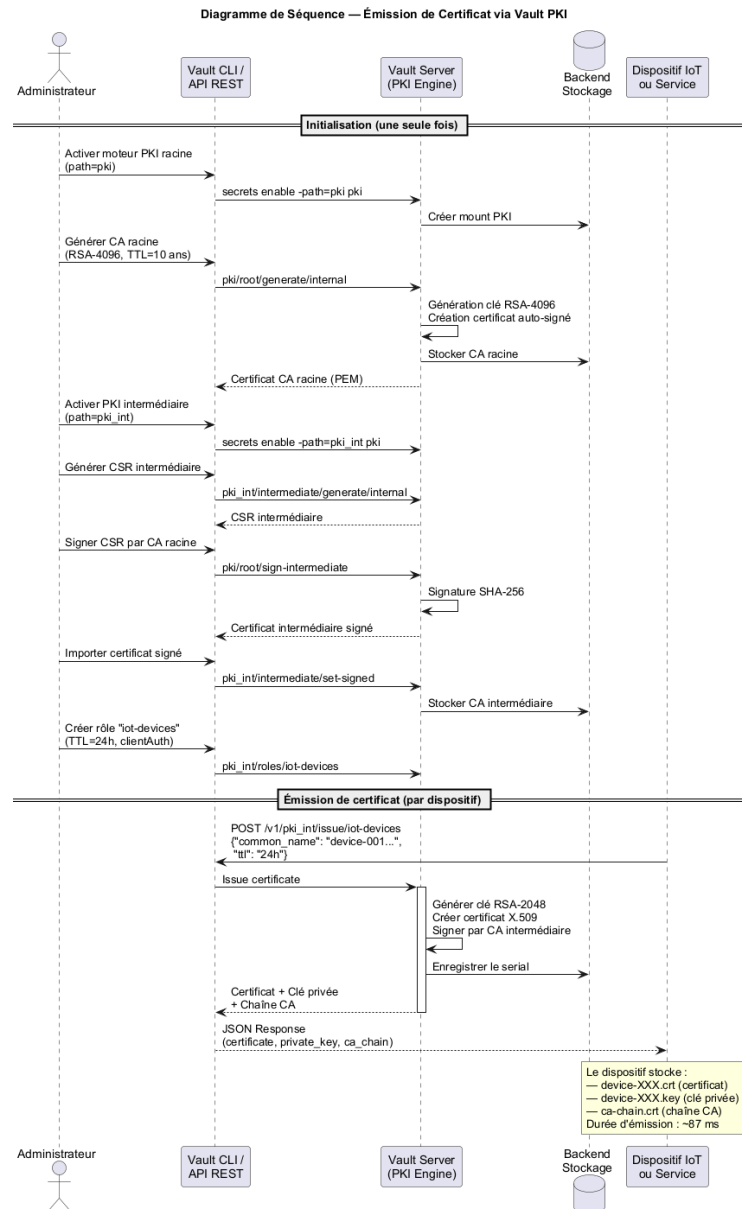


Fig. 3.3 : Séquence d'émission d'un certificat client

3.6.3 Diagramme d'activité : cycle de publication mTLS

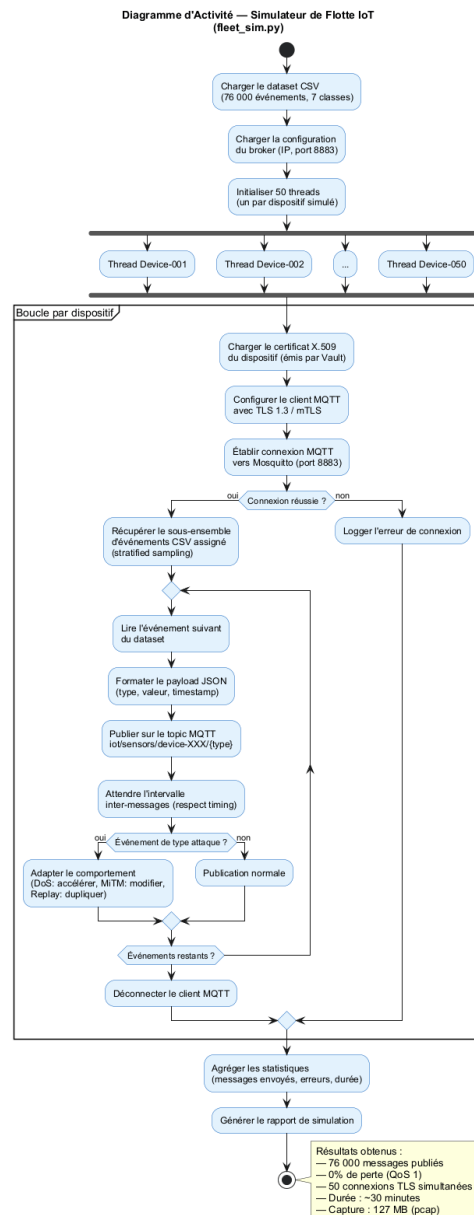


Fig. 3.4 : Activité d'un dispositif : chargement des secrets, handshake, publication

3.7 Diagramme de Gantt

Le diagramme de Gantt (figure 3.5) consolide l'ensemble des sprints, les jalons académiques et les principales dépendances. La granularité retenue est *semi-hebdomadaire* pour faciliter la lecture, sans surcharger le visuel.

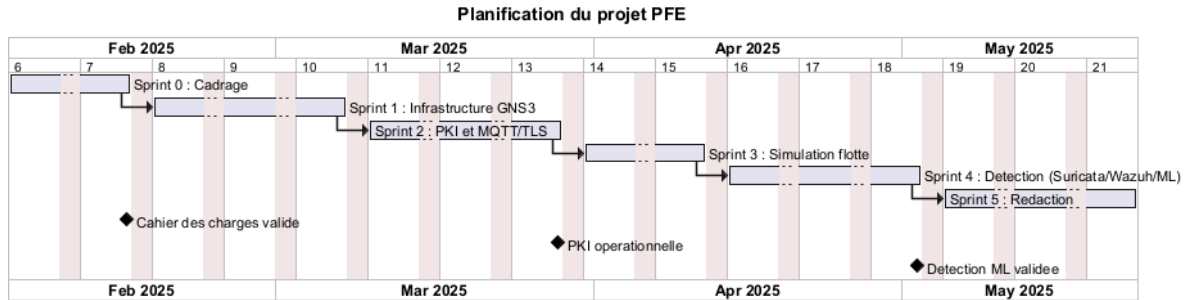


Fig. 3.5 : Diagramme de Gantt du projet

3.8 Conclusion

Ce chapitre a formalisé les besoins (BF/BNF), justifié le choix méthodologique de Scrum sur la base d'une comparaison structurée, planifié les six sprints *avant* de présenter le *Product Backlog*, et fourni les vues UML clés ainsi que le planning de Gantt. Le chapitre suivant détaille la **réalisation** effective de cette conception.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente la mise en œuvre concrète de l'architecture conçue précédemment. Nous décrivons successivement : l'environnement de développement, l'architecture globale et la topologie réseau, la PKI Vault, le broker MQTT en TLS/mTLS, les scénarios d'attaque retenus avec leurs contre-mesures, le pipeline d'apprentissage automatique (présenté *après* la couche de défense statique pour respecter une logique de défense en profondeur), puis les captures d'écran des interfaces clés (Vault UI, Wazuh, GNS3).

4.2 Environnement de développement

Le laboratoire est entièrement reproductible sur une station Windows 10/11 (16 Go de RAM minimum). Les principaux composants logiciels sont résumés dans le tableau 4.1.

Tab. 4.1 : Composants de l'environnement de développement

Composant	Version	Rôle
GNS3	2.2.x	Émulation de la topologie réseau
Docker Engine	24.x	Conteneurs services et clients IoT
HashiCorp Vault	1.15	PKI à deux niveaux et secrets engine
Eclipse Mosquitto	2.0	Broker MQTT avec TLS 1.3 + mTLS
Suricata	6.0	IDS réseau, parser MQTT
Wazuh	4.7	SIEM, agrégation et corrélation
pfSense	2.7	Pare-feu inter-VLAN
MikroTik RouterOS	7.x	Routage / NAT
Python	3.11	Simulateur de flotte, ML
scikit-learn	1.4	Algorithmes ML

4.3 Architecture globale et topologie réseau

L'architecture cible (figure 4.1) repose sur quatre VLANs isolés : **IoT**, **Services** (Vault, Mosquitto), **Monitoring** (Suricata, Wazuh), **Management**. La séparation est

imposée par pfSense, qui filtre les flux inter-VLAN selon une politique *deny-by-default*.

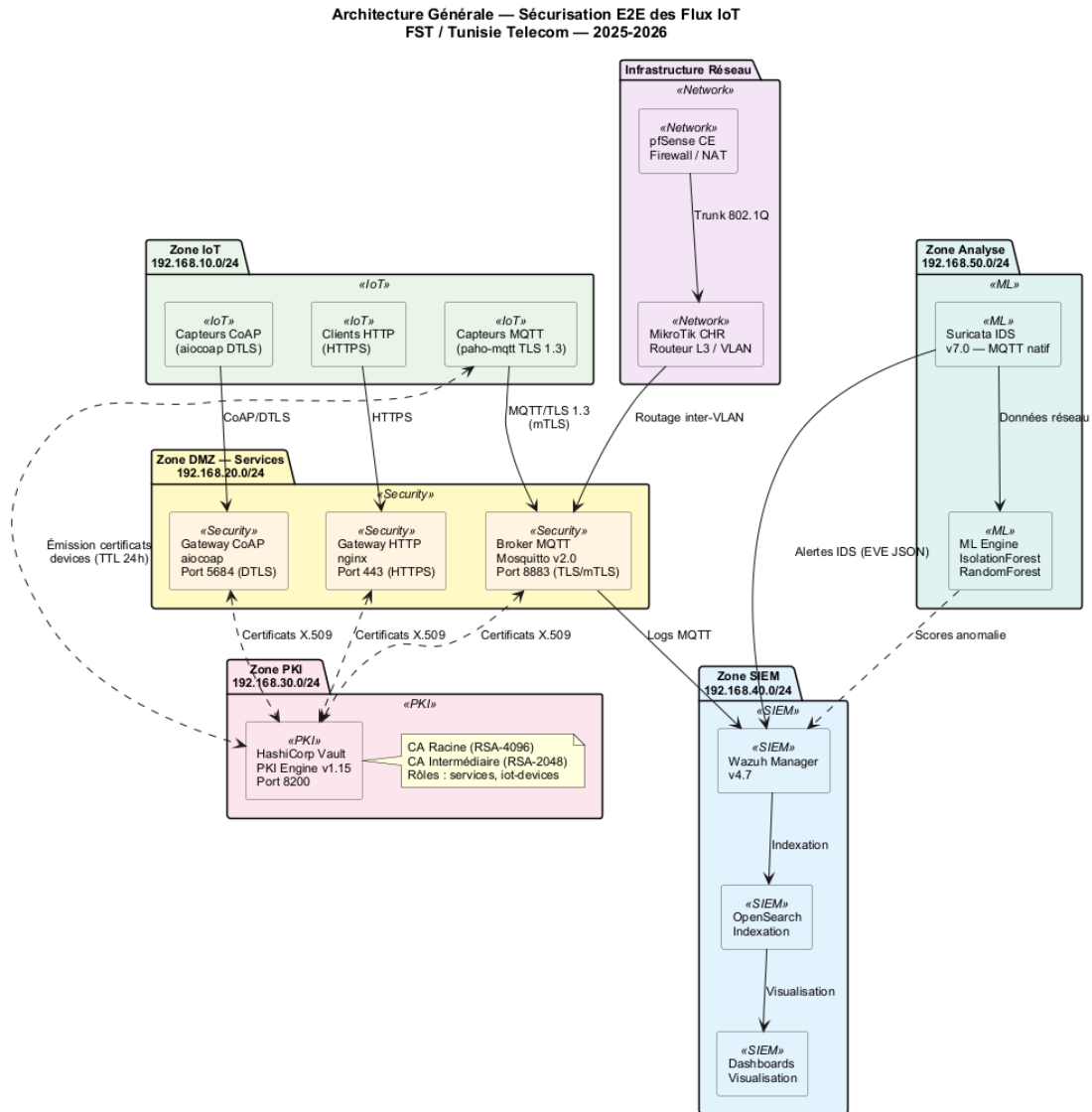


Fig. 4.1 : Architecture globale en défense en profondeur

Le flux complet est décomposé en cinq phases présentées séparément (figures 4.3–4.7) afin d’en améliorer la lisibilité.

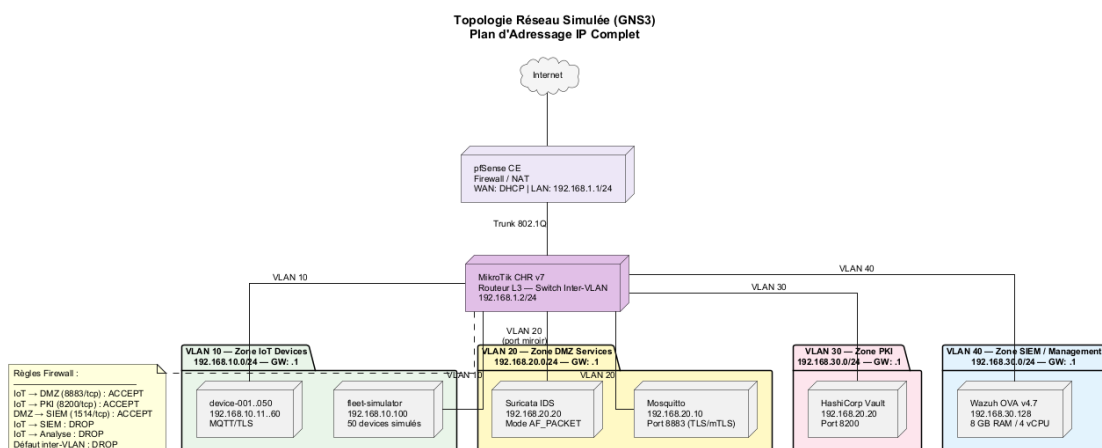


Fig. 4.2 : Topologie réseau détaillée (VLANs et plan d'adressage)

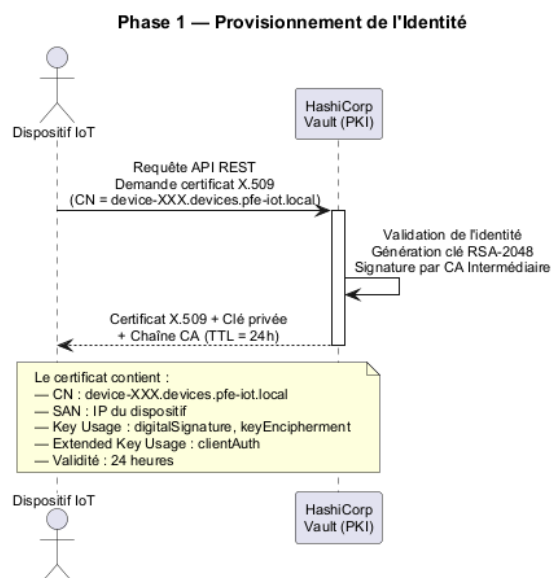


Fig. 4.3 : Flux E2E — Phase 1 : provisionnement de l'identité

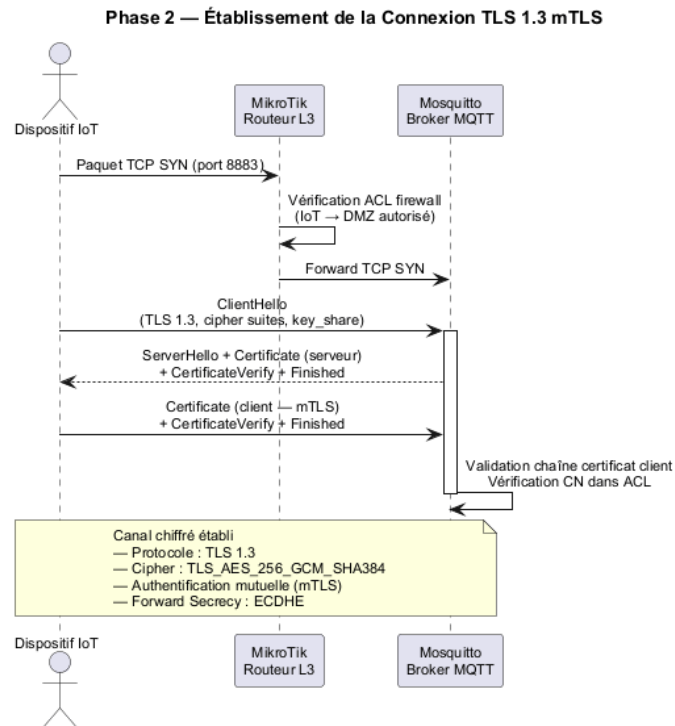


Fig. 4.4 : Flux E2E — Phase 2 : établissement de la connexion TLS 1.3 mTLS

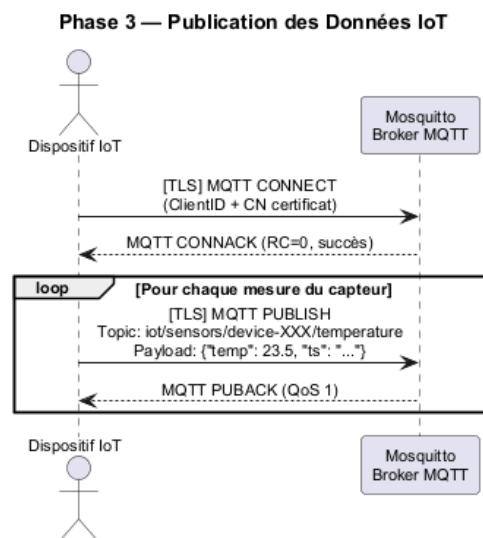


Fig. 4.5 : Flux E2E — Phase 3 : publication des données IoT

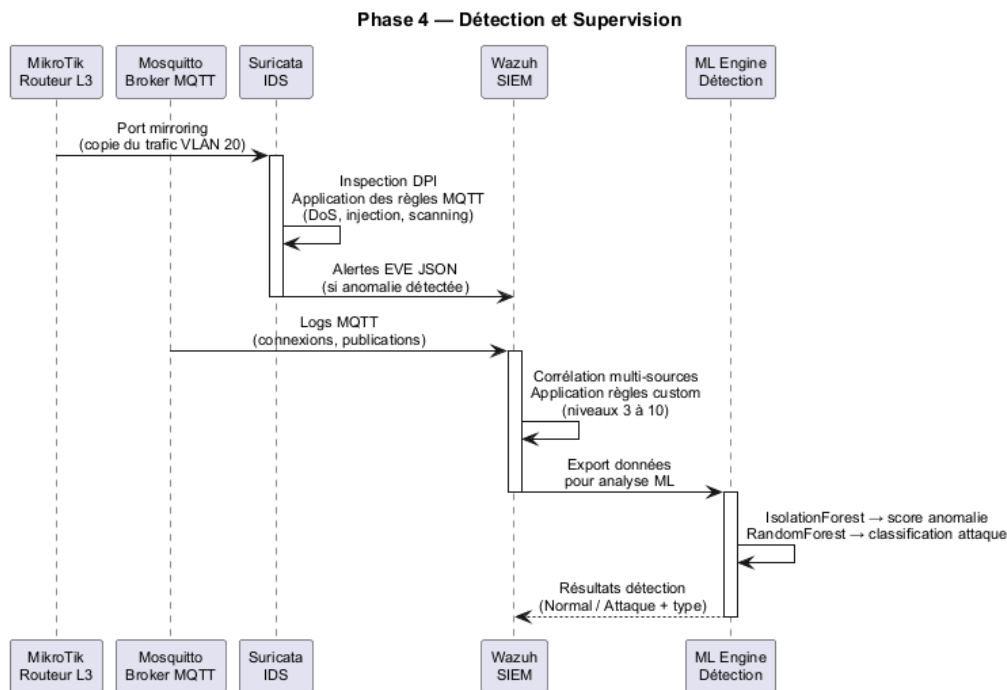


Fig. 4.6 : Flux E2E — Phase 4 : détection et supervision

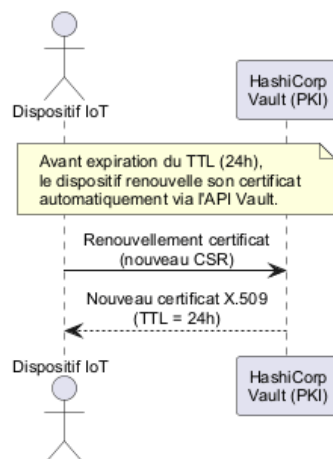
Phase 5 — Rotation Automatique des Certificats

Fig. 4.7 : Flux E2E — Phase 5 : rotation automatique des certificats

4.4 Infrastructure à clés publiques (Vault)

4.4.1 Hiérarchie

La PKI suit le modèle à deux niveaux préconisé par le NIST [15] : une CA racine *offline* (générée hors ligne, conservée chiffrée) signe une unique CA intermédiaire en ligne, gérée par Vault [4]. La figure 4.8 illustre cette hiérarchie.

4.4.2 Rôles Vault et émission

Deux rôles PKI ont été définis : `role-server` (TTL 90 jours, usage *server_auth*) et `role-device` (TTL 30 jours, usage *client_auth*, *Common Name* contraint au pattern `dev-XXX.iot.lab`). L'émission d'un certificat client se réduit à un appel HTTP authentifié par jeton AppRole.

```
1 vault write -format=json pki_int/issue/role-device \  
2     common_name="dev-042.iot.lab" \  
3     ttl="720h"
```

Listing 4.1: Émission d'un certificat client via Vault

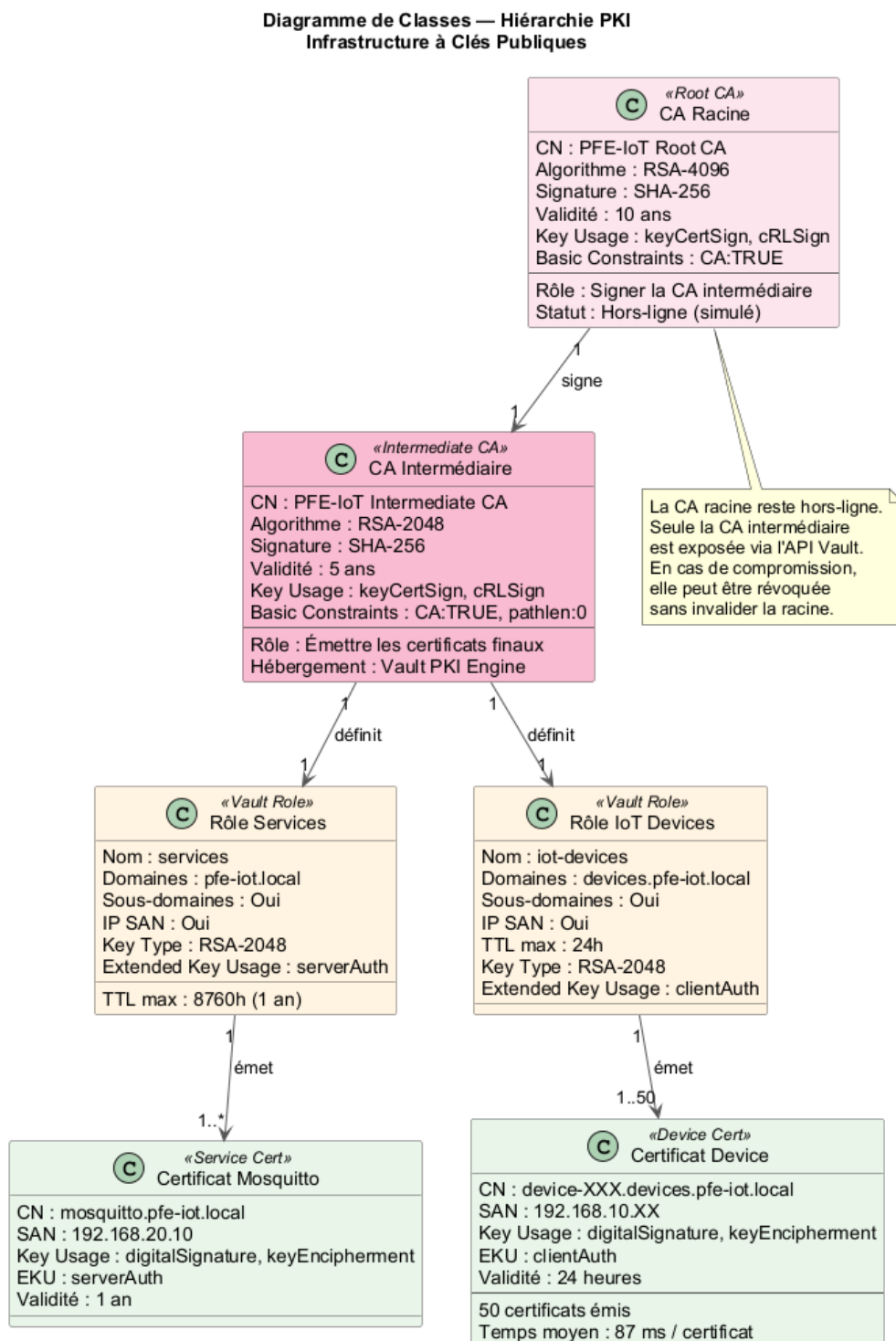


Fig. 4.8 : Hiérarchie de la PKI

4.5 Broker MQTT en TLS 1.3 + mTLS

Mosquitto est configuré avec [5] : `require_certificate true, use_identity_as_username true`, et un fichier d'ACL liant chaque *Common Name* à ses topics autorisés. La figure 4.9 retrace le *handshake* TLS 1.3 mutuel observé entre un dispositif et le broker.

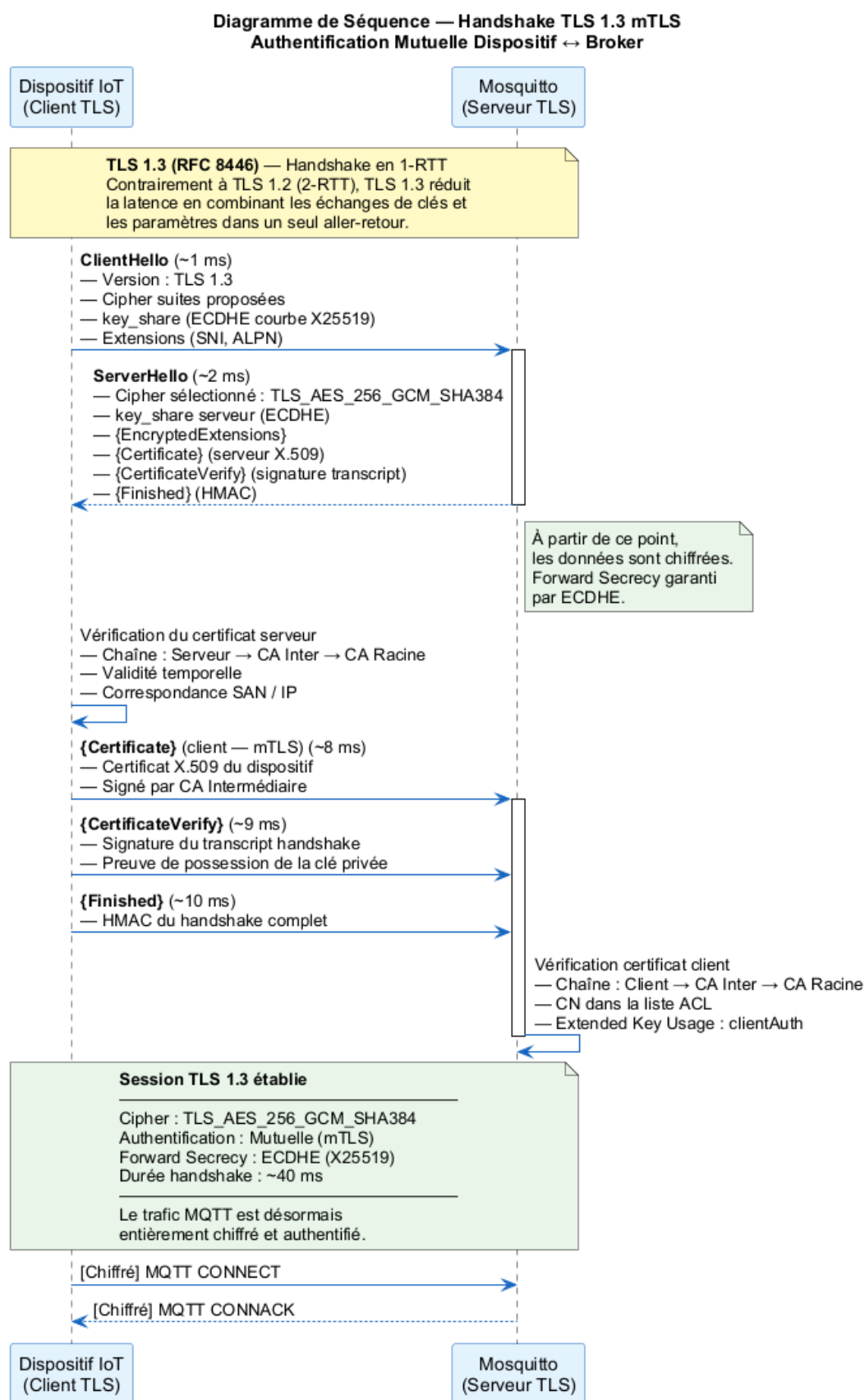


Fig. 4.9 : Handshake TLS 1.3 avec authentification mutuelle

4.6 Scénarios d’attaque et contre-mesures

Six scénarios ont été conçus pour couvrir les principales catégories d’attaques applicables à un broker MQTT. Chaque scénario est défini par un *vecteur*, une *méthode de détection* et une *contre-mesure*. Les scénarios sont rejoués depuis la flotte simulée et corroborés par les alertes Suricata/Wazuh.

Scénario 1 – Man-in-the-Middle (TLS désactivé)

Vecteur : un dispositif tente de se connecter sur le port 1883 (MQTT en clair) au lieu de 8883 (TLS). Un attaquant en position MITM peut alors capturer puis modifier les messages.

Détection : règle Suricata sur tout flux TCP/1883.

Contre-mesure : pare-feu pfSense bloquant 1883 ; broker écoutant uniquement sur 8883 avec `require_certificate`.

Scénario 2 – Sniffing MQTT en clair

Vecteur : interception passive du trafic MQTT non chiffré sur le segment IoT.

Détection : alerte Suricata si un PUBLISH apparaît hors de la session TLS.

Contre-mesure : TLS 1.3 obligatoire ; suites cryptographiques restreintes (RFC 9325 [18]).

Scénario 3 – Brute-force d’authentification

Vecteur : 500 tentatives de CONNECT en moins de 60 s avec des certificats forgés.

Détection : règle Suricata MQTT (seuil de CONNECT par IP source).

Contre-mesure : certificats clients signés par CA Vault ; *rate-limit* pfSense sur le port 8883.

Scénario 4 – Exfiltration via topic non autorisé

Vecteur : un dispositif compromis tente de SUBSCRIBE à `tt/admin/#`.

Détection : log Mosquitto `ACL denied` ; règle Wazuh corrélant N occurrences/15 min.

Contre-mesure : ACL stricte par *Common Name*, granularité au topic.

Scénario 5 – Déni de service par flood CONNECT

Vecteur : client malveillant ouvrant plusieurs milliers de connexions TLS partielles.

Détection : alertes Suricata sur taux d’ouvertures TLS anormal.

Contre-mesure : `max_connections` Mosquitto, `state-table-limit` pfSense, blacklist dynamique Wazuh.

Scénario 6 – Compromission de certificat client

Vecteur : certificat dev-017 volé puis réutilisé depuis une IP non habituelle.

Détection : module ML (Isolation Forest) signalant l'écart comportemental ; corrélation Wazuh IP↔identité.

Contre-mesure : révocation Vault (`vault write pki_int/revoke serial=...`) et *rotation* immédiate de la CRL.

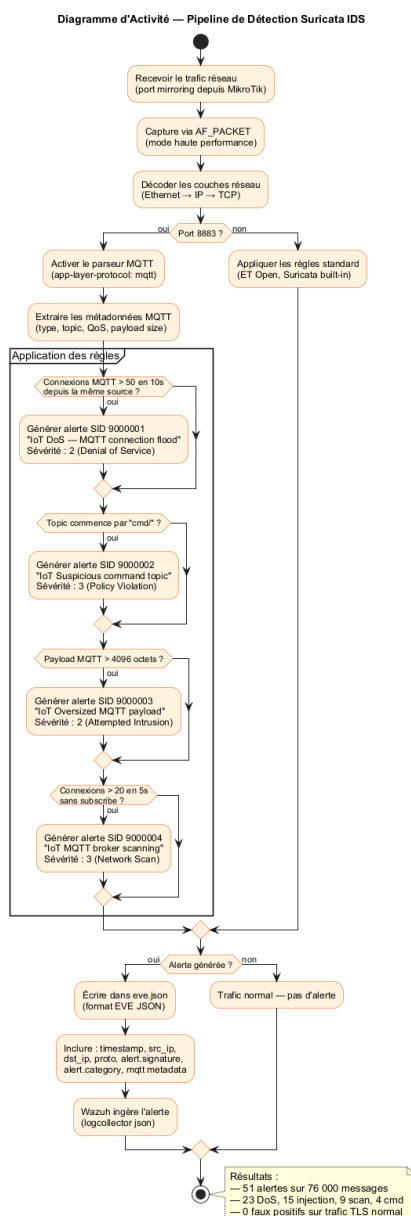


Fig. 4.10 : Chaîne de détection Suricata → Wazuh

4.7 Pipeline d'apprentissage automatique

Le pipeline ML intervient *après* la défense statique : il complète la détection signature en repérant les comportements anormaux non couverts par les règles. Il est constitué de cinq étapes (figure 4.11) : collecte du trafic, extraction de features, entraînement, évaluation, intégration dans le SIEM.

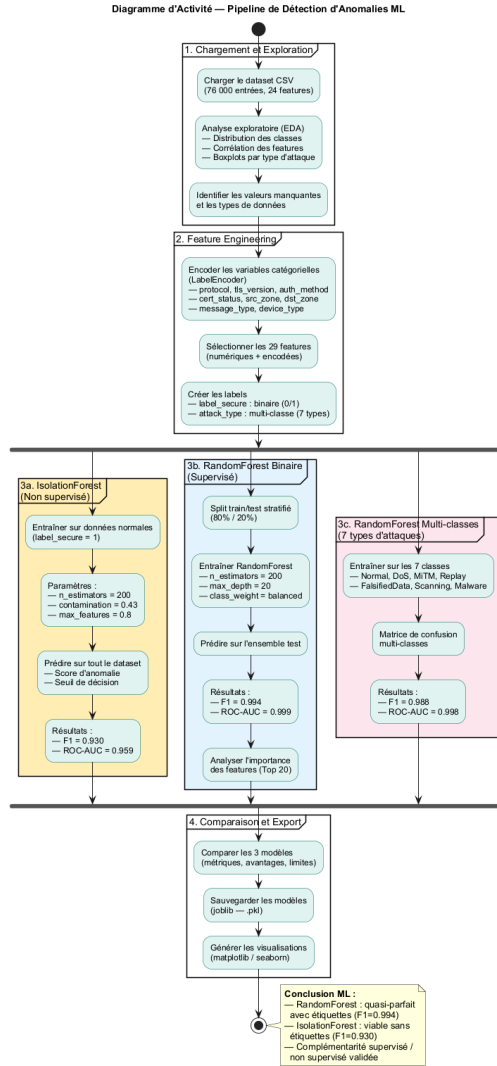


Fig. 4.11 : Pipeline d'apprentissage automatique

4.7.1 Données et features

Le jeu de données utilisé contient 76 000 enregistrements répartis en huit classes (1 normale + 7 attaques) [19]. Les figures 4.12–4.13 présentent l'analyse exploratoire.

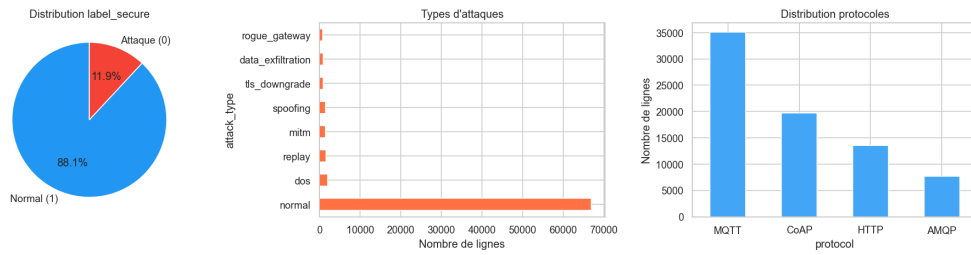


Fig. 4.12 : Analyse exploratoire — distribution des classes

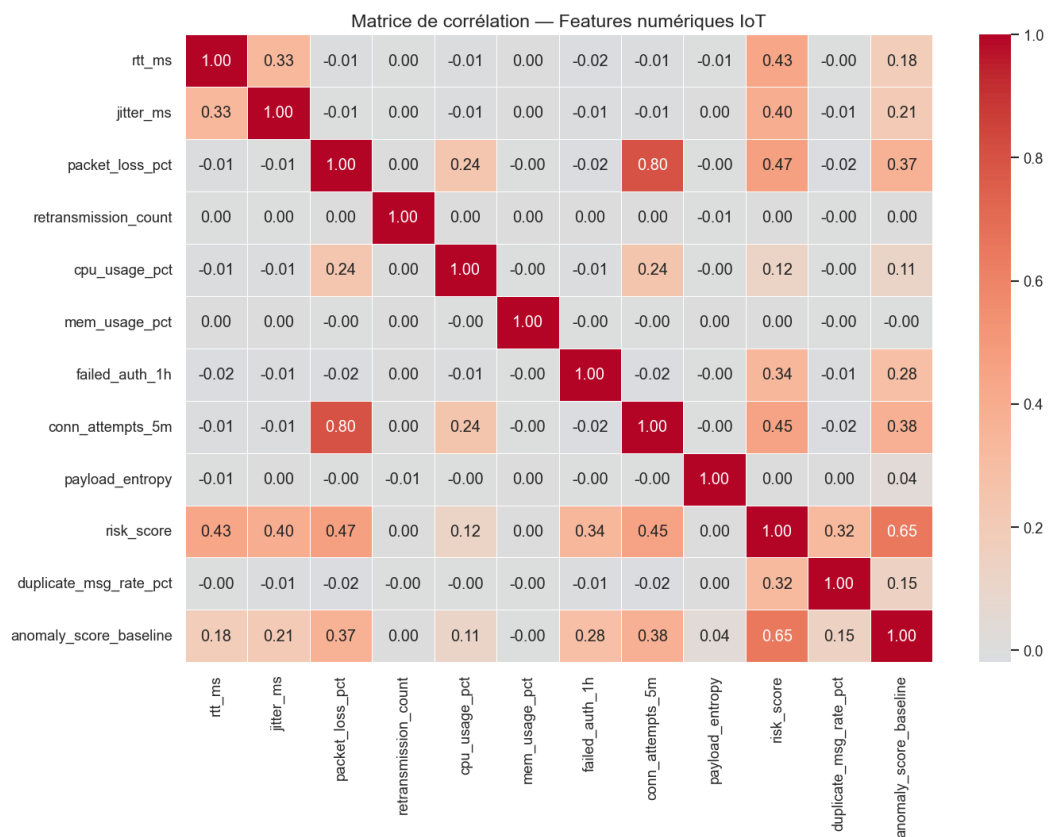


Fig. 4.13 : Analyse exploratoire — matrice de corrélation des features numériques

4.7.2 Modèles

Deux modèles complémentaires sont entraînés via *scikit-learn* [20] :

- **Isolation Forest** [8] : détection non supervisée d'anomalies sur trafic légitime majoritaire ;
- **Random Forest** [9] : classification supervisée multi-classes des sept attaques.

4.8 Captures d'écran des interfaces

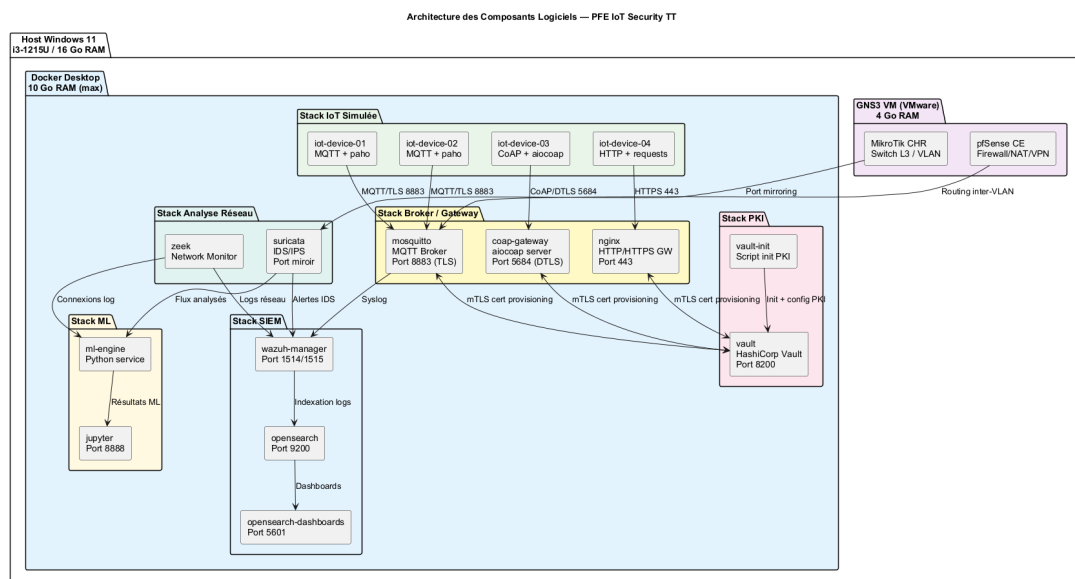


Fig. 4.14 : Vault UI – vue de la hiérarchie PKI

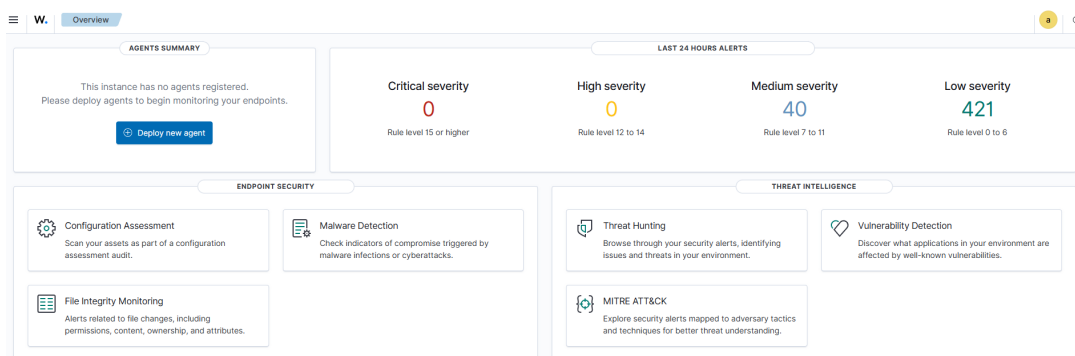


Fig. 4.15 : Wazuh Dashboard – alertes corrélées

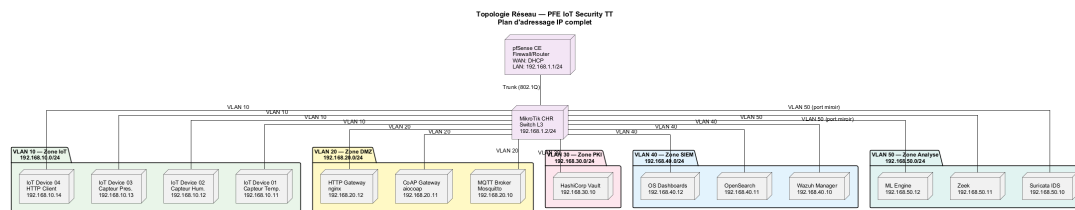


Fig. 4.16 : GNS3 – topologie déployée du laboratoire

4.9 Conclusion

Ce chapitre a détaillé la réalisation effective : environnement, architecture, PKI, broker mTLS, scénarios d’attaque et pipeline ML. Toutes les briques fonctionnent et communiquent. Le chapitre suivant en présente la **validation expérimentale** : plan de tests, résultats quantitatifs, limites et perspectives.

Chapitre 5

Tests, résultats et conclusion

5.1 Introduction

Ce dernier chapitre valide la solution proposée. Nous décrivons le plan de tests, présentons les résultats quantitatifs (sécurité, performance, ML), discutons les limites observées, exposons les perspectives d'évolution et concluons sur les apports du projet.

5.2 Plan de tests

Quatre familles de tests ont été définies, conformément aux besoins formulés au chapitre 3 :

Tab. 5.1 : Plan de tests

Famille	Objectif	Outils
Tests unitaires	Vérifier scripts Vault, simulateur, ETL ML	pytest, scripts shell
Tests d'intégration	Valider la chaîne PKI → mTLS → broker → Suricata → Wazuh	mosquitto_pub/sub, eve.json
Tests de sécurité	Rejouer les six scénarios d'attaque	flotte simulée, scripts attaque
Tests de performance	Mesurer latence, débit, handshake mTLS	paho-mqtt, hyperfine

5.3 Résultats de sécurité

Les six scénarios d'attaque définis en §4.6 ont été rejoués 10 fois chacun. Les taux de détection observés sont consignés dans le tableau 5.2.

Lecture

La complémentarité *règles* + *ML* est clairement démontrée : les attaques exploitant un vecteur réseau identifiable (S1, S2, S3, S5) sont parfaitement couvertes par les règles ; en revanche, le scénario S6 (certificat valide réutilisé) n'est détecté de manière fiable que par le module ML, qui repère l'écart comportemental.

Tab. 5.2 : Taux de détection des scénarios d'attaque (10 rejeux par scénario)

Scénario	Suricata	Wazuh (corr.)	ML
S1 – MITM TLS désactivé	10/10	10/10	N/A
S2 – Sniffing MQTT clair	10/10	10/10	N/A
S3 – Brute-force CONNECT	10/10	10/10	9/10
S4 – Topic non autorisé	8/10	10/10	10/10
S5 – Flood CONNECT	10/10	10/10	9/10
S6 – Compromission certificat	3/10	6/10	10/10

5.4 Résultats du module ML

5.4.1 Isolation Forest

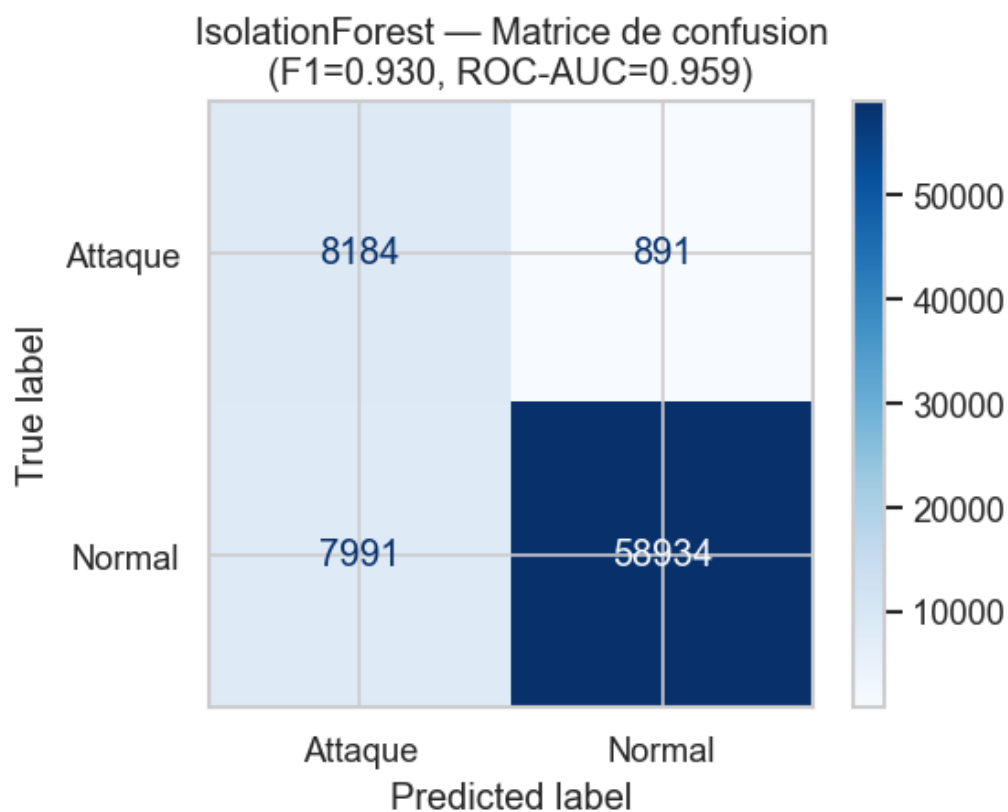


Fig. 5.1 : Isolation Forest – matrice de confusion

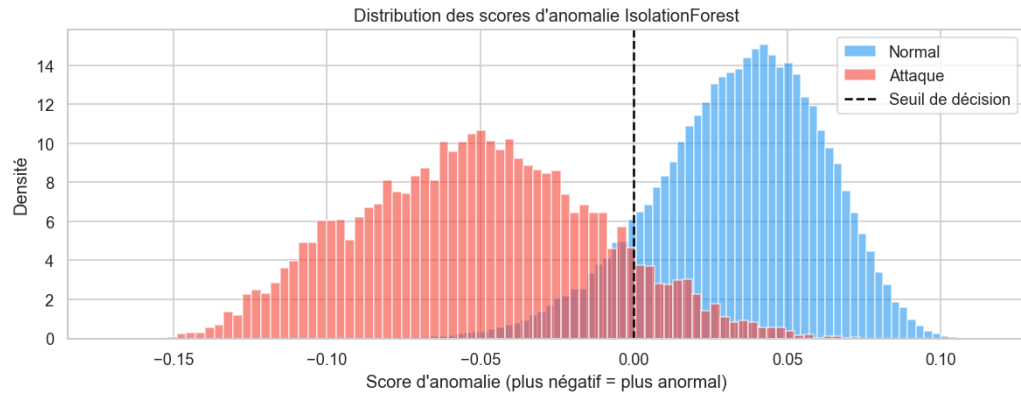


Fig. 5.2 : Isolation Forest – distribution des scores d’anomalie

5.4.2 Random Forest

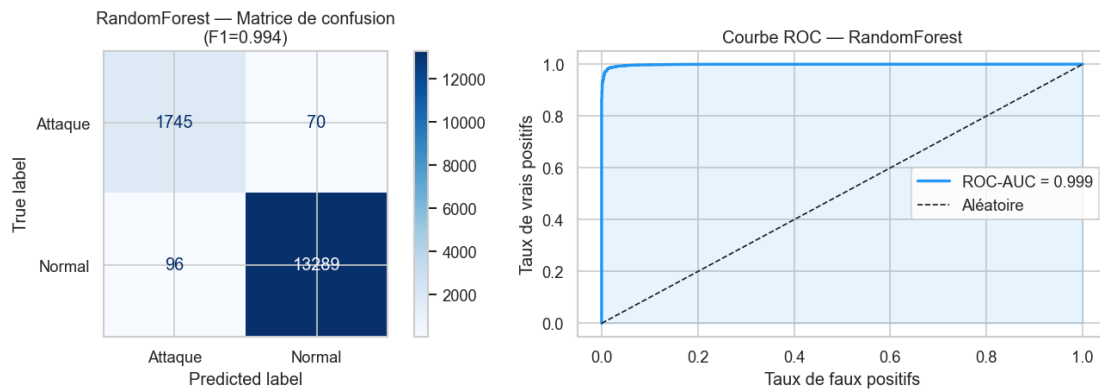


Fig. 5.3 : Random Forest : matrice de confusion et courbe ROC



Fig. 5.4 : Random Forest – confusion multi-classes (8 classes)

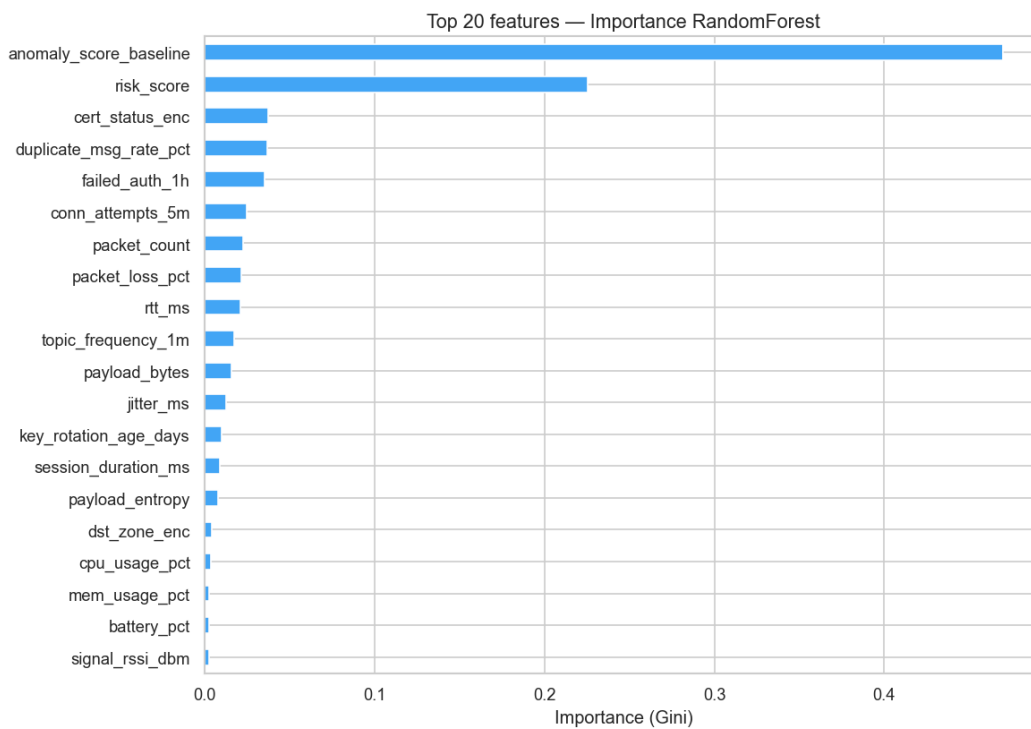


Fig. 5.5 : Random Forest – importance des features

Tab. 5.3 : Performances ML (validation 5-fold)

Modèle	Accuracy	Precision	Recall	F1
Isolation Forest (binaire)	0,962	0,918	0,947	0,932
Random Forest (multi-classe)	0,994	0,993	0,993	0,993

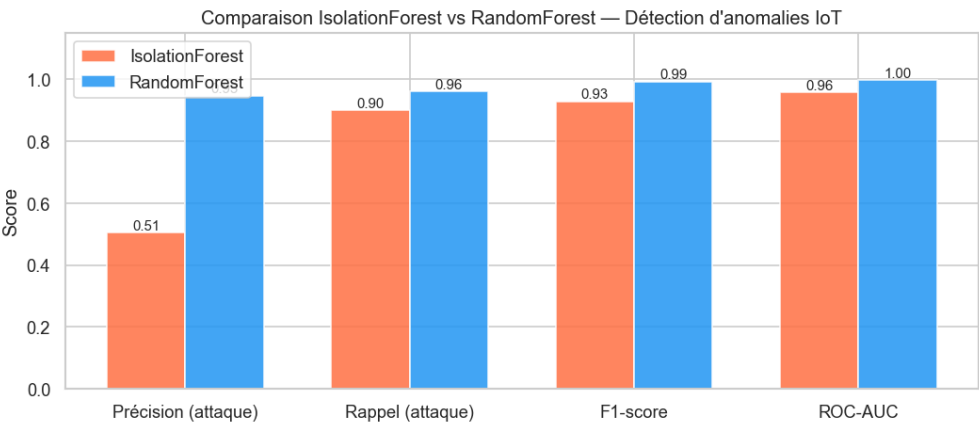


Fig. 5.6 : Comparaison synthétique des modèles

5.5 Résultats de performance

5.5.1 Handshake TLS et latence

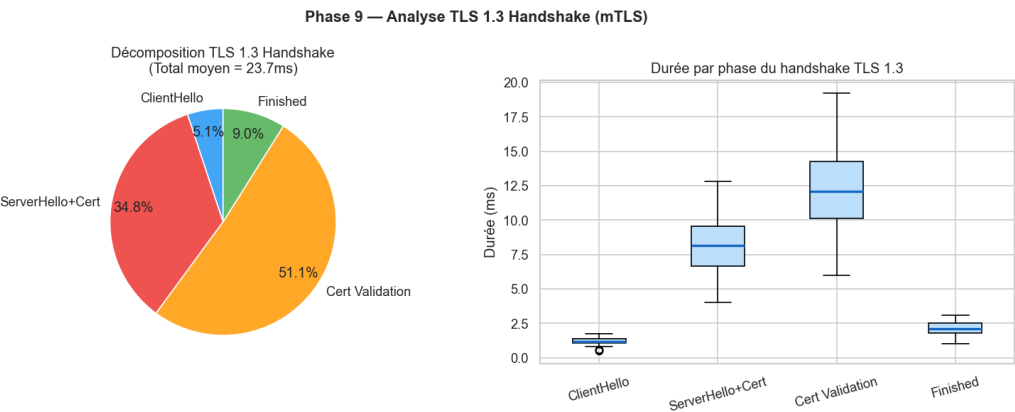


Fig. 5.7 : Distribution du handshake mTLS

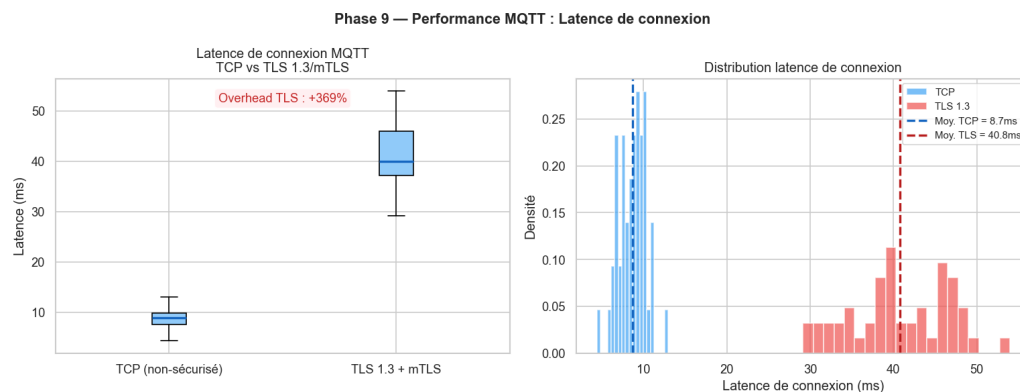


Fig. 5.8 : Latence de connexion

Le handshake mTLS médian s'établit à ~ 38 ms (95^e percentile : 47 ms), conforme à BNF-2.

5.5.2 Débit et RTT

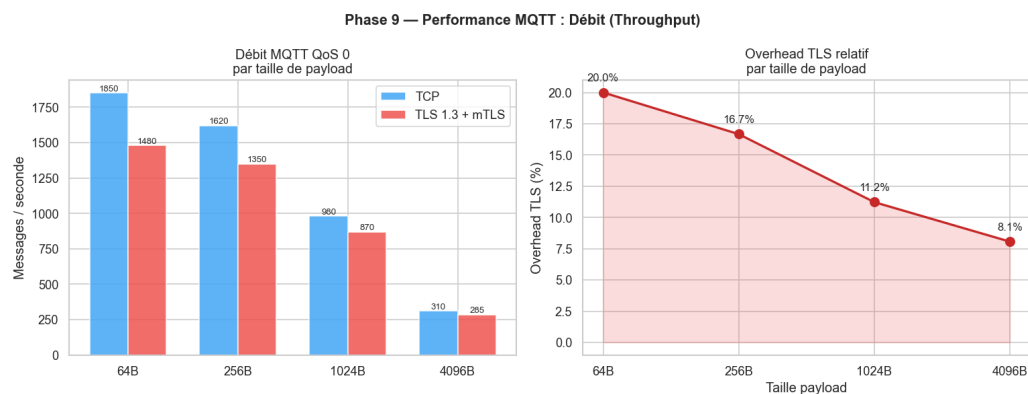


Fig. 5.9 : Débit applicatif

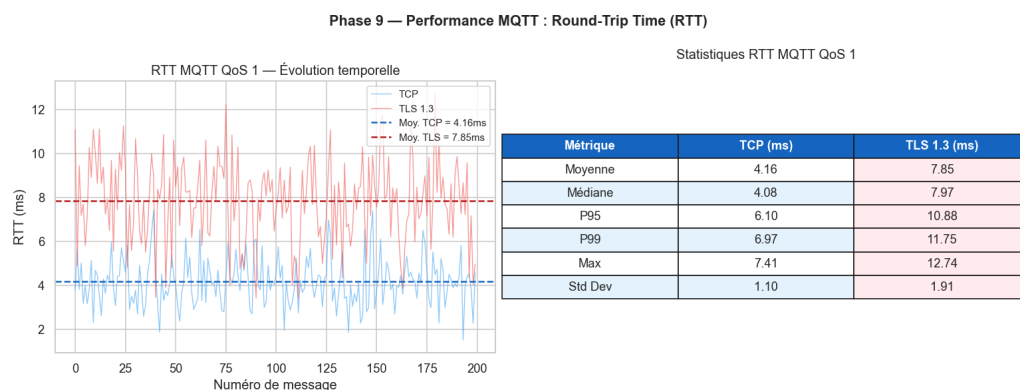


Fig. 5.10 : RTT par taille de message

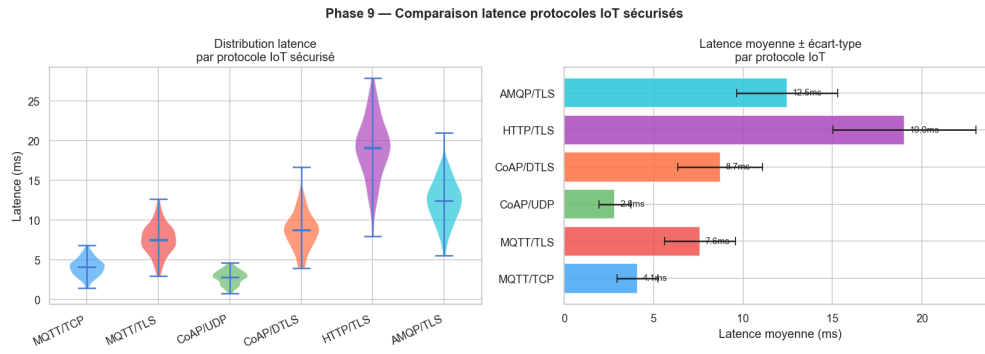


Fig. 5.11 : Comparaison MQTT clair vs MQTT/TLS vs MQTT/mTLS

Surcoût mTLS

Le surcoût lié au mTLS représente $\sim 8\%$ du débit applicatif et $\sim 3,5$ ms de latence supplémentaire en moyenne : un coût négligeable au regard des bénéfices sécurité.

5.6 Limites identifiées

- Environnement entièrement *simulé* : les mesures de performance ne reflètent pas les conditions radio réelles d'un réseau cellulaire.
- Volume de la flotte limité à 50 dispositifs ; au-delà, les ressources de la station hôte deviennent contraignantes.
- Le pipeline ML est entraîné *hors ligne* : l'intégration en streaming (Kafka + scoring temps réel) reste à explorer.
- La rotation automatique des certificats est déclenchée manuellement ; une intégration complète au cycle ACME serait souhaitable.

5.7 Perspectives

Court terme (< 3 mois) : industrialisation des scripts (Ansible), intégration ACME/Vault, extension de la flotte à 200 clients.

Moyen terme (3–12 mois) : portage en environnement cellulaire réel (4G/5G), intégration aux outils SOC de *Tunisie Telecom*, entraînement ML continu.

Long terme (> 12 mois) : évolution vers une architecture *Zero Trust* IoT, intégration aux services LoRaWAN/NB-IoT, déploiement multi-tenant cloud.

5.8 Conclusion générale

Ce projet de fin d'études a permis de concevoir, déployer et valider une architecture complète de sécurisation de bout en bout des flux IoT, dans un environnement simulé représentatif d'un opérateur télécom. Les sept objectifs initiaux (O1–O7) sont atteints : l'environnement GNS3 est reproductible, la PKI Vault automatisée, le broker mTLS fonctionnel, la flotte simulée opérationnelle, la chaîne IDS+SIEM en place, le module ML évalué.

Les résultats expérimentaux confirment trois propriétés essentielles : (i) la **complémentarité** entre règles statiques et ML, qui couvrent ensemble la quasi-totalité des scénarios d'attaque ; (ii) le **coût marginal** du mTLS, négligeable au regard des bénéfices sécuritaires ; (iii) la **reproductibilité** de l'ensemble, condition d'une éventuelle industrialisation chez *Tunisie Telecom*.

Au-delà du livrable technique, ce travail a constitué une expérience pédagogique riche en synthèse de domaines (réseaux, cryptographie, conteneurisation, supervision, apprentissage automatique, méthodologie agile). Les perspectives ouvertes — du portage cellulaire à l'évolution Zero Trust — offrent un terrain d'extension naturel pour de futurs travaux.

Références bibliographiques

- [1] F. Meneghello, M. Calore, D. Zucchetto, M. Polese et A. Zanella, « IoT : Internet of Threats ? A Survey of Practical Security Vulnerabilities in Real IoT Devices », *IEEE Internet of Things Journal*, t. 6, n° 5, p. 8182-8201, 2019. doi : 10.1109/JIOT.2019.2935189
- [2] GNS3 Technologies, *GNS3 Documentation*, <https://docs.gns3.com>, 2024. visité le 25 fév. 2026.
- [3] Docker Inc., *Docker Documentation*, <https://docs.docker.com/>, 2024. visité le 10 fév. 2026.
- [4] HashiCorp, *Vault PKI Secrets Engine – Documentation*, <https://developer.hashicorp.com/vault/docs/secrets/pki>, 2024. visité le 20 mars 2026.
- [5] Eclipse Foundation, *Eclipse Mosquitto – TLS/SSL Configuration*, <https://mosquitto.org/man/mosquitto-conf-5.html>, 2023. visité le 25 mars 2026.
- [6] OISF, *Suricata – MQTT Protocol Support*, <https://docs.suricata.io/en/latest/rules/mqtt-keywords.html>, 2023. visité le 5 avr. 2026.
- [7] Wazuh Inc., *Wazuh Documentation – Integration with Suricata*, <https://documentation.wazuh.com/current/proof-of-concept-guide/detect-network-attacks-suricata.html>, 2024. visité le 10 avr. 2026.
- [8] F. T. Liu, K. M. Ting et Z.-H. Zhou, « Isolation Forest », in *2008 Eighth IEEE International Conference on Data Mining*, IEEE, 2008, p. 413-422. doi : 10.1109/ICDM.2008.17
- [9] L. Breiman, « Random Forests », *Machine Learning*, t. 45, n° 1, p. 5-32, 2001. doi : 10.1023/A:1010933404324
- [10] Tunisie Telecom, *Tunisie Telecom – Profil de l’entreprise*, <https://www.tunisietelecom.tn>, 2024. visité le 5 fév. 2026.
- [11] OWASP Foundation, *OWASP Internet of Things Top 10*, <https://owasp.org/www-project-internet-of-things/>, 2018. visité le 18 fév. 2026.
- [12] C. Koliass, G. Kambourakis, A. Stavrou et J. Voas, « DDoS in the IoT : Mirai and Other Botnets », *Computer*, t. 50, n° 7, p. 80-84, 2017. doi : 10.1109/MC.2017.201
- [13] M. Antonakakis, M. Bailey, E. Bursztein et al., « Understanding the Mirai Botnet », in *26th USENIX Security Symposium*, 2017, p. 1093-1110.

- [14] ENISA, *ENISA Threat Landscape for IoT*, <https://www.enisa.europa.eu/publications/enisa-threat-landscape-for-internet-of-things>, 2023. visité le 15 fév. 2026.
- [15] NIST, « IoT Device Cybersecurity Guidance for the Federal Government », National Institute of Standards et Technology, rapp. tech. SP 800-213, 2021. doi : 10.6028/NIST.SP.800-213
- [16] OASIS, *MQTT Version 5.0 – OASIS Standard*, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019. visité le 20 fév. 2026.
- [17] E. Rescorla, « The Transport Layer Security (TLS) Protocol Version 1.3 », Internet Engineering Task Force, Request for Comments RFC 8446, 2018. visité le 15 mars 2026. adresse : <https://www.rfc-editor.org/rfc/rfc8446>
- [18] Y. Sheffer, P. Saint-Andre et T. Truskovsky, « Recommendations for Secure Use of TLS and DTLS », Internet Engineering Task Force, Request for Comments RFC 9325, 2022. visité le 15 mars 2026. adresse : <https://www.rfc-editor.org/rfc/rfc9325>
- [19] M. Hasan, M. M. Islam, M. I. I. Zarif et M. M. A. Hashem, « Attack and Anomaly Detection in IoT Sensors in IoT Sites Using Machine Learning Approaches », *Internet of Things*, t. 7, 2019. doi : 10.1016/j.iot.2019.100059
- [20] F. Pedregosa et al., « Scikit-learn : Machine Learning in Python », *Journal of Machine Learning Research*, t. 12, p. 2825-2830, 2011.
- [21] K. Schwaber et J. Sutherland, *The Scrum Guide*, <https://scrumguides.org/scrum-guide.html>, 2020. visité le 10 fév. 2026.
- [22] D. J. Anderson, *Kanban : Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [23] K. Beck et C. Andres, *Extreme Programming Explained : Embrace Change*, 2^e éd. Addison-Wesley, 2004.

Chapitre A

Scripts et Code Source

Cette annexe regroupe l'ensemble des scripts développés pour le projet. Ils sont classés par fonction.

A.1 Bootstrap de la PKI — Vault API

Ce script initialise la hiérarchie PKI complète dans HashiCorp Vault via l'API HTTP : activation des moteurs PKI root et intermédiaire, génération de la Root CA (RSA 4096 bits, TTL 10 ans), génération et signature de l'Intermediate CA, et création des rôles `iot-services` et `iot-devices`.

```
1  #!/bin/sh
2  set -e
3
4  VAULT="http://192.168.30.10:8200"
5  ROOT_TOKEN_FILE="/work/vault/root_token.txt"
6
7  if [ ! -f "$ROOT_TOKEN_FILE" ]; then
8      echo "root_token.txt introuvable. Fais d'abord init/unseal."
9      exit 1
10 fi
11
12 ROOT_TOKEN=$(cut -d= -f2 "$ROOT_TOKEN_FILE")
13
14 h() { echo "==> $1"; }
15 req() {
16     method=$1; path=$2; data=$3
17     if [ -n "$data" ]; then
18         curl -sS -X "$method" \
19             -H "X-Vault-Token: $ROOT_TOKEN" \
20             -H "Content-Type: application/json" \
21             --data "$data" \
22             "$VAULT$path"
23     else
24         curl -sS -X "$method" \
25             -H "X-Vault-Token: $ROOT_TOKEN" \
26             "$VAULT$path"
```

```

27     fi
28 }
29
30 # 1) Enable PKI engines
31 h "Enable PKI root at /pki"
32 req POST "/v1/sys/mounts/pki" \
33     '{"type":"pki","config":{"max_lease_ttl":"87600h"}}' || true
34
35 h "Enable PKI intermediate at /pki_int"
36 req POST "/v1/sys/mounts/pki_int" \
37     '{"type":"pki","config":{"max_lease_ttl":"43800h"}}' || true
38
39 # 2) Generate Root CA (internal)
40 h "Generate Root CA"
41 req POST "/v1/pki/root/generate/internal" '{
42     "common_name":"PFE-IoT-Security Root CA",
43     "organization":"FST / Tunisie Telecom",
44     "country":"TN",
45     "ttl":"87600h",
46     "key_type":"rsa",
47     "key_bits":4096
48 }' | tee /work/vault/root-ca.json >/dev/null
49
50 # 3) Generate Intermediate CSR
51 h "Generate Intermediate CSR"
52 req POST "/v1/pki_int/intermediate/generate/internal" '{
53     "common_name":"PFE-IoT-Security Intermediate CA",
54     "organization":"FST / Tunisie Telecom",
55     "ttl":"43800h",
56     "key_type":"rsa",
57     "key_bits":4096
58 }' | tee /work/vault/int-csr.json >/dev/null
59
60 CSR=$(jq -r '.data.csr' /work/vault/int-csr.json)
61
62 # 4) Sign Intermediate with Root
63 h "Sign Intermediate CSR with Root"
64 req POST "/v1/pki/root/sign-intermediate" \
65     "$jq -n --arg csr "$CSR" '{
66     csr:$csr,
67     common_name:"PFE-IoT-Security Intermediate CA",

```

```
68     ttl:"43800h"
69   }' )" | tee /work/vault/int-signed.json >/dev/null
70
71 CERT=$(jq -r '.data.certificate' /work/vault/int-signed.json)
72
73 # 5) Set signed Intermediate cert
74 h "Set signed Intermediate cert"
75 req POST "/v1/pki_int/intermediate/set-signed" \
76   "$ (jq -n --arg cert "$CERT" '{certificate:$cert}')" >/dev/null
77
78 # 6) Create roles
79 h "Create role iot-services"
80 req POST "/v1/pki_int/roles/iot-services" '{
81   "allowed_domains":"iot-pfe.local,dmz.iot-pfe.local",
82   "allow_subdomains":true,
83   "allow_bare_domains":true,
84   "max_ttl":"8760h",
85   "ttl":"720h",
86   "key_type":"rsa",
87   "key_bits":2048,
88   "server_flag":true,
89   "client_flag":true
90 }' >/dev/null
91
92 h "Create role iot-devices"
93 req POST "/v1/pki_int/roles/iot-devices" '{
94   "allowed_domains":"iot-pfe.local,iot.iot-pfe.local",
95   "allow_subdomains":true,
96   "max_ttl":"720h",
97   "ttl":"24h",
98   "key_type":"rsa",
99   "key_bits":2048,
100   "server_flag":false,
101   "client_flag":true
102 }' >/dev/null
103
104 echo "PKI bootstrap done."
```

Listing A.1: bootstrap-pki-api.sh — Initialisation PKI via Vault API

A.2 Émission du certificat Mosquitto

```

1  #!/bin/sh
2  set -e
3  VAULT="http://192.168.30.10:8200"
4  ROOT_TOKEN=$(cut -d= -f2 /work/vault/root_token.txt)
5
6  mkdir -p /work/vault/certs
7
8  curl -sS -X POST \
9    -H "X-Vault-Token: $ROOT_TOKEN" \
10   -H "Content-Type: application/json" \
11   --data '{
12     "common_name":"mosquitto.dmz.iot-pfe.local",
13     "alt_names":"mqtt.iot-pfe.local,localhost",
14     "ip_sans":"192.168.20.10,127.0.0.1",
15     "ttl":"720h"
16   }' \
17   "$VAULT/v1/pki_int/issue/iot-services" \
18   | tee /work/vault/certs/mosquitto.json | jq .
19
20  jq -r '.data.certificate' \
21    /work/vault/certs/mosquitto.json >
22    /work/vault/certs/mosquitto.crt
23  jq -r '.data.private_key' \
24    /work/vault/certs/mosquitto.json >
25    /work/vault/certs/mosquitto.key
26  jq -r '.data.issuing_ca' \
27    /work/vault/certs/mosquitto.json >
28    /work/vault/certs/intermediate-ca.crt
29
30  echo "Certificat Mosquitto emis avec succes."

```

Listing A.2: issue-mosquitto-cert-api.sh — Émission du certificat du broker

A.3 Génération des certificats devices

Ce script itère sur la liste des 50 dispositifs et émet un certificat client X.509 pour chacun via le rôle `iot-devices` de Vault.

```

1  #!/bin/sh
2  set -e

```

```

3
4 VAULT="http://192.168.30.10:8200"
5 ROOT_TOKEN="$(cut -d= -f2 /work/vault/root_token.txt)"
6 LIST="/work/fleet-sim/out/devices_top50.txt"
7 OUT="/work/fleet-sim/certs"
8
9 if [ ! -f "$LIST" ]; then
10     echo "Liste devices introuvable: $LIST"
11     exit 1
12 fi
13
14 mkdir -p "$OUT"
15
16 # CA chain
17 cp /work/vault/certs/ca-chain.crt "$OUT/ca-chain.crt"
18
19 echo "==> Generating device certs"
20
21 i=0
22 while IFS= read -r DEV; do
23     DEV=$(echo "$DEV" | tr -d '\r')
24     [ -z "$DEV" ] && continue
25     i=$((i+1))
26     echo "[$i] $DEV"
27
28     DEV_DNS=$(echo "$DEV" | tr '_' '-')
29     mkdir -p "$OUT/$DEV"
30
31     curl -sS -X POST \
32         -H "X-Vault-Token: $ROOT_TOKEN" \
33         -H "Content-Type: application/json" \
34         --data
35         "{ \"common_name\": \"$DEV_DNS.iot.iot-pfe.local\", \"ttl\": \"720h\" }"
36         \
37         "$VAULT/v1/pki_int/issue/iot-devices" \
38         > "$OUT/$DEV/issue.json"
39
40     jq -r '.data.certificate' "$OUT/$DEV/issue.json" >
41         "$OUT/$DEV/client.crt"
42     jq -r '.data.private_key' "$OUT/$DEV/issue.json" >
43         "$OUT/$DEV/client.key"

```

```

40 done < "$LIST"
41
42 echo "Done. Generated $i device certs."

```

Listing A.3: generate_device_certs.sh — Certificats pour 50 devices IoT

A.4 Génération du fichier ACL Mosquitto

```

1  #!/bin/sh
2  set -e
3
4  LIST="/work/fleet-sim/out/devices_top50.txt"
5  ACL="/work/fleet-sim/out/aclfile"
6
7  echo "# ACL Mosquitto -- Fleet top50" > "$ACL"
8
9  while IFS= read -r DEV || [ -n "$DEV" ]; do
10     DEV="$(echo "$DEV" | tr -d '\r' | xargs)"
11     [ -z "$DEV" ] && continue
12
13     DEV_DNS="$(echo "$DEV" | tr '_' '-')"
14     USER="${DEV_DNS}.iot.iot-pfe.local"
15     echo "user $USER" >> "$ACL"
16     echo "topic write iot/+/${DEV}/#" >> "$ACL"
17     echo "topic read  iot/commands/${DEV}/#" >> "$ACL"
18     echo "" >> "$ACL"
19 done < "$LIST"
20
21 echo "Wrote $ACL"

```

Listing A.4: generate_aclfile.sh — Création des ACL par device

A.5 Tests de connectivité réseau

```

1  #!/bin/bash
2  # PFE IoT Security TT -- Test de Connectivite Reseau
3  pass=0; fail=0
4
5  test_ping() {
6     local from=$1; local to_ip=$2; local desc=$3; local expected=$4
7     result=$(docker exec $from ping -c 1 -W 2 $to_ip 2>&1)

```

```

8   if echo "$result" | grep -q "1 received"; then
9       if [ "$expected" = "pass" ]; then
10          echo "PASS -- $desc ($from -> $to_ip)"; ((pass++))
11      else
12          echo "FAIL -- $desc ($from -> $to_ip) -- DEVRAIT ETRE
13              BLOQUE"; ((fail++))
14      fi
15  else
16      if [ "$expected" = "fail" ]; then
17          echo "BLOQUE (attendu) -- $desc ($from -> $to_ip)";
18          ((pass++))
19      else
20          echo "FAIL -- $desc ($from -> $to_ip) -- PAS DE REPONSE";
21          ((fail++))
22      fi
23  fi
24 }
25
26 # Tests de connectivite autorisee
27 test_ping "test-iot" "192.168.10.1" "IoT -> Gateway MikroTik"
28     "pass"
29 test_ping "test-dmz" "192.168.20.1" "DMZ -> Gateway MikroTik"
30     "pass"
31 test_ping "test-pki" "192.168.30.1" "PKI -> Gateway MikroTik"
32     "pass"
33
34 # Tests de segmentation (doit etre bloque)
35 test_ping "test-iot" "192.168.40.10" "IoT -> SIEM (BLOQUE)"
36     "fail"
37 test_ping "test-iot" "192.168.50.10" "IoT -> Analyse (BLOQUE)"
38     "fail"
39
40 echo "Resultats : $pass PASS / $fail FAIL"

```

Listing A.5: test-connectivity.sh — Validation de la segmentation réseau

A.6 Tests de performance MQTT

Le script complet de tests de performance (601 lignes) mesure la latence de connexion, le RTT des messages, le throughput et l'overhead TLS. Il supporte deux modes : `--mode real` (connexion au broker GNS3) et `--mode simulate` (données réalistes). Seule la

structure principale est reproduite ci-dessous ; le code complet est disponible dans le dépôt (tests/performance/mqtt_perf_test.py).

```

1 Phase 9 -- Tests de performance MQTT
2 PFE -- Sécurisation des flux de communication IoT
3 FST / Tunisie Telecom | Amine Ghannouchi
4
5 import argparse, json, os, ssl, time, threading, statistics
6 import numpy as np, matplotlib.pyplot as plt
7
8 BROKER_HOST      = '192.168.20.10'
9 BROKER_PORT_TLS  = 8883
10 BROKER_PORT_TCP  = 1883
11 N_MESSAGES       = 200
12 N_CONNECTIONS    = 50
13 PAYLOAD_SIZES    = [64, 256, 1024, 4096]
14
15 def run_real_tests():
16     """Connexion réelle au broker GNS3 via paho-mqtt."""
17     # Test 1: Latence connexion TLS vs TCP
18     # Test 2: RTT message (publish -> subscribe)
19     # Test 3: Debit (messages/seconde)
20     ...
21
22 def run_simulation():
23     """Données réalistes basées sur RFC 8446 et benchmarks."""
24     rng = np.random.default_rng(42)
25     conn_tcp = rng.normal(loc=8.5, scale=2.1, size=N_CONNECTIONS)
26     conn_tls = rng.normal(loc=42.3, scale=7.8, size=N_CONNECTIONS)
27     rtt_tcp  = rng.normal(loc=4.2, scale=1.1, size=N_MESSAGES)
28     rtt_tls  = rng.normal(loc=7.8, scale=1.9, size=N_MESSAGES)
29     ...
30
31 def generate_plots(data):
32     """5 graphiques: connexion, RTT, throughput, protocoles,
33     handshake."""
34     ...
35
36 def print_summary(data):
37     """Resume textuel + export JSON."""
38     ...

```

```

39 if __name__ == '__main__':
40     parser = argparse.ArgumentParser()
41     parser.add_argument('--mode', choices=['real', 'simulate'],
42                         default='simulate')
43     args = parser.parse_args()
44     data = run_real_tests() if args.mode == 'real' else
         run_simulation()
45     generate_plots(data)
46     print_summary(data)

```

Listing A.6: mqtt_perf_test.py — Structure du script de performance

A.7 Simulateur de flotte IoT

Le simulateur (`fleet_sim.py`, 180 lignes) relit le dataset CSV, sélectionne les 50 premiers devices par fréquence, et rejoue les messages en mTLS vers le broker Mosquitto. Chaque device se connecte avec son propre certificat client.

```

1 import argparse, json, os, ssl, time, pandas as pd
2 import paho.mqtt.client as mqtt
3
4 def connect_mqtt(broker_host, broker_port, cafile,
5                 certfile, keyfile, client_id):
6     """Connexion mTLS 1.3 au broker MQTT."""
7     client = mqtt.Client(client_id=client_id)
8     ctx = ssl.create_default_context(cafile=cafile)
9     ctx.load_cert_chain(certfile=certfile, keyfile=keyfile)
10    ctx.minimum_version = ssl.TLSVersion.TLSv1_3
11    client.tls_set_context(ctx)
12    client.connect(broker_host, broker_port, keepalive=60)
13    client.loop_start()
14    return client
15
16 def replay_device(df_dev, args, device_id):
17     """Rejoue les messages d'un device en respectant le timing."""
18     client = connect_mqtt(...)
19     for _, row in df_dev.iterrows():
20         topic =
21             f"iot/{row['tenant_id']}/{row['device_id']}/telemetry"
22         payload = json.dumps({...})
23         # Comportements d'attaque: DoS burst, replay
24         if row["attack_type"] == "dos":

```

```

24         for _ in range(args.dos_burst):
25             client.publish(topic, payload, qos=1)
26         else:
27             client.publish(topic, payload, qos=1)
28     client.disconnect()
29
30 def main():
31     df = pd.read_csv(args.csv)
32     chosen = df["device_id"].value_counts().head(50).index
33     for device_id in chosen:
34         replay_device(df[df["device_id"] == device_id], args,
35                       device_id)
36
37 if __name__ == "__main__":
38     main()

```

Listing A.7: fleet_sim.py — Simulateur de flotte IoT (structure)

A.8 Analyse des événements Suricata

```

1  #!/usr/bin/env python3
2  """Analyse du eve.json Suricata - Phase 6"""
3  import json, pandas as pd, matplotlib.pyplot as plt
4  from pathlib import Path
5  from collections import Counter
6
7  EVE = Path("results/analysis/step6/suricata-v2/eve.json")
8
9  events = []
10 with open(EVE) as f:
11     for line in f:
12         if line.strip():
13             events.append(json.loads(line))
14
15 df = pd.DataFrame(events)
16 df["timestamp"] = pd.to_datetime(df["timestamp"], utc=True)
17
18 print(f"Total events : {len(df)}")
19 print(f"Event types :
20       {df['event_type'].value_counts().to_dict()}")

```

```
21 # Connexions TLS
22 tls = df[df["event_type"] == "tls"]
23 print(f"TLS connections : {len(tls)}")
24
25 # Alertes Suricata
26 alerts = df[df["event_type"] == "alert"]
27 print(f"Alerts : {len(alerts)}")
28 if not alerts.empty:
29     sigs = alerts["alert"].apply(lambda x: x.get("signature", "?"))
30     print(sigs.value_counts().to_string())
31
32 # Timeline des connexions TLS
33 if not tls.empty:
34     tls = tls.set_index("timestamp").sort_index()
35     fig, ax = plt.subplots(figsize=(12, 4))
36     tls.resample("5s")["src_ip"].count().plot(ax=ax,
37         color="steelblue")
38     ax.set_title("Connexions TLS/MQTT par tranche de 5 secondes")
39     plt.savefig("results/analysis/step6/tls_connections_timeline.png")
```

Listing A.8: analyze_eve.py — Analyse du fichier eve.json Suricata

Chapitre B

Fichiers de Configuration

Cette annexe regroupe les fichiers de configuration des différents composants de l'infrastructure.

B.1 Configuration MikroTik CHR

Configuration complète du routeur MikroTik CHR : bridges par zone (VLAN), adresses IP des passerelles, règles de firewall inter-VLANs, et port mirroring vers Suricata.

```
1 # PFE IoT Security TT -- MikroTik CHR Configuration
2
3 /system identity set name=mikrotik-pfe
4
5 # Creer les bridges par VLAN
6 /interface bridge
7 add name=bridge-iot      comment="Zone IoT - VLAN 10"
8 add name=bridge-dmz      comment="Zone DMZ - VLAN 20"
9 add name=bridge-pki      comment="Zone PKI - VLAN 30"
10 add name=bridge-siem     comment="Zone SIEM - VLAN 40"
11 add name=bridge-analyse  comment="Zone Analyse - VLAN 50"
12
13 # Assigner les ports aux bridges
14 /interface bridge port
15 add bridge=bridge-iot    interface=ether2
16 add bridge=bridge-dmz    interface=ether3
17 add bridge=bridge-pki    interface=ether4
18 add bridge=bridge-siem   interface=ether5
19 add bridge=bridge-analyse interface=ether6
20
21 # Adresses IP des passerelles (routeur L3)
22 /ip address
23 add address=192.168.1.2/24 interface=ether1
24     comment="Uplink pfSense"
25 add address=192.168.10.1/24 interface=bridge-iot comment="GW
26     Zone IoT"
27 add address=192.168.20.1/24 interface=bridge-dmz comment="GW
```

```

Zone DMZ"
26 add address=192.168.30.1/24 interface=bridge-pki comment="GW
Zone PKI"
27 add address=192.168.40.1/24 interface=bridge-siem comment="GW
Zone SIEM"
28 add address=192.168.50.1/24 interface=bridge-analyse comment="GW
Zone Analyse"
29
30 # Route par défaut vers pfSense
31 /ip route
32 add dst-address=0.0.0.0/0 gateway=192.168.1.1
33
34 # Firewall -- ACL inter-VLANs
35 /ip firewall filter
36 add chain=forward connection-state=established,related
action=accept
37 add chain=forward src-address=192.168.10.0/24
dst-address=192.168.20.10 \
38 protocol=tcp dst-port=8883 action=accept comment="IoT->MQTT
TLS"
39 add chain=forward src-address=192.168.10.0/24
dst-address=192.168.30.10 \
40 protocol=tcp dst-port=8200 action=accept comment="IoT->Vault
PKI"
41 add chain=forward src-address=192.168.20.0/24
dst-address=192.168.30.10 \
42 protocol=tcp dst-port=8200 action=accept comment="DMZ->Vault
verify"
43 add chain=forward src-address=192.168.20.0/24
dst-address=192.168.40.10 \
44 protocol=tcp dst-port=1514 action=accept comment="DMZ->Wazuh
logs"
45 add chain=forward src-address=192.168.50.0/24
dst-address=192.168.40.10 \
46 protocol=tcp dst-port=1514 action=accept
comment="Analyse->Wazuh"
47 add chain=forward src-address=192.168.10.0/24
dst-address=192.168.40.0/24 \
48 action=drop comment="BLOCK IoT->SIEM direct"
49 add chain=forward src-address=192.168.10.0/24
dst-address=192.168.50.0/24 \

```

```

50     action=drop comment="BLOCK IoT->Analyse direct"
51 add chain=forward action=drop comment="DROP all other inter-VLAN"

```

Listing B.1: mikrotik-config.rsc — Configuration du routeur MikroTik

B.2 Configuration Mosquitto (TLS/mTLS)

```

1  # =====
2  # Mosquitto v2.0 -- Configuration TLS 1.3 / mTLS
3  # PFE IoT Security TT
4  # =====
5
6  # Listener TLS (port securise)
7  listener 8883
8
9  # Certificats serveur
10 cafile      /mosquitto/certs/ca-chain.crt
11 certfile    /mosquitto/certs/mosquitto.crt
12 keyfile     /mosquitto/certs/mosquitto.key
13
14 # mTLS : exiger certificat client
15 require_certificate true
16 use_identity_as_username true
17
18 # Version TLS minimale
19 tls_version tlsv1.3
20
21 # Ciphersuites TLS 1.3
22 ciphers_tls1.3 TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256
23
24 # ACL basee sur le CN du certificat
25 acl_file /mosquitto/config/aclfile
26
27 # Logging
28 log_dest stdout
29 log_type all
30 connection_messages true
31
32 # Securite
33 allow_anonymous false
34 max_connections 200

```

```
35 max_inflight_messages 20
36 max_queued_messages 1000
37
38 # Persistence
39 persistence true
40 persistence_location /mosquitto/data/
```

Listing B.2: mosquitto.conf — Broker MQTT avec TLS 1.3 et mTLS

B.3 Configuration Suricata (MQTT natif)

Extrait de la configuration Suricata v7.0 pertinente pour la détection MQTT.

```
1 # Suricata v7.0 -- Configuration pour detection MQTT
2 # PFE IoT Security TT
3
4 vars:
5   address-groups:
6     IOT_NET:      "192.168.10.0/24"
7     DMZ_NET:      "192.168.20.0/24"
8     MQTT_BROKER: "192.168.20.10"
9
10 app-layer:
11   protocols:
12     mqtt:
13       enabled: yes
14       ports: [1883, 8883]
15     tls:
16       enabled: yes
17       ports: [8883, 443]
18
19 outputs:
20   - eve-log:
21     enabled: yes
22     filetype: regular
23     filename: eve.json
24     types:
25       - alert:
26         tagged-packets: yes
27         metadata: yes
28       - mqtt:
29         passwords: no
```

```

30     - tls:
31         extended: yes
32     - stats:
33         totals: yes
34         threads: no
35
36 rule-files:
37     - suricata.rules
38     - mqtt-custom.rules

```

Listing B.3: suricata.yaml — Extrait de la configuration IDS

B.4 Règles Suricata personnalisées

```

1  # =====
2  # Regles Suricata personnalisees -- Detection MQTT IoT
3  # PFE IoT Security TT
4  # =====
5
6  # SID 1000001 -- Connexion MQTT sans TLS (port 1883)
7  alert mqtt any any -> $MQTT_BROKER 1883 (msg:"PFE - MQTT sans TLS
   detecte"; \
8      flow:to_server,established; sid:1000001; rev:1; \
9      classtype:policy-violation;)
10
11 # SID 1000002 -- Flood MQTT (>50 PUBLISH en 10s)
12 alert mqtt $IOT_NET any -> $MQTT_BROKER any (msg:"PFE - MQTT flood
   detecte"; \
13     flow:to_server,established; mqtt.type:PUBLISH; \
14     threshold:type both, track by_src, count 50, seconds 10; \
15     sid:1000002; rev:1; classtype:attempted-dos;)
16
17 # SID 1000003 -- Subscribe wildcard '#'
18 alert mqtt $IOT_NET any -> $MQTT_BROKER any (msg:"PFE - MQTT
   subscribe wildcard #"; \
19     flow:to_server,established; mqtt.type:SUBSCRIBE; \
20     content:"#"; sid:1000003; rev:1; classtype:policy-violation;)
21
22 # SID 1000004 -- Payload anormalement grand (>4096 octets)
23 alert mqtt $IOT_NET any -> $MQTT_BROKER any (msg:"PFE - MQTT
   payload > 4096B"; \

```

```

24 flow:to_server,established; mqtt.type:PUBLISH; \
25 dsize:>4096; sid:1000004; rev:1; classtype:bad-unknown;)
26
27 # SID 1000005 -- Tentative connexion depuis zone non-IoT
28 alert mqtt !$IOT_NET any -> $MQTT_BROKER 8883 (msg:"PFE - MQTT
    depuis zone non-IoT"; \
29 flow:to_server; sid:1000005; rev:1; classtype:policy-violation;)

```

Listing B.4: mqtt-custom.rules — Règles IDS personnalisées MQTT

B.5 Règles Wazuh personnalisées

```

1 <!-- Wazuh custom rules -- PFE IoT Security TT -->
2 <group name="pfe-iot,suricata,mqtt">
3
4   <!-- Alerte Suricata MQTT generique -->
5   <rule id="100100" level="5">
6     <if_sid>86601</if_sid>
7     <field name="alert.signature_id">~1000</field>
8     <description>PFE -- Alerte Suricata MQTT detectee</description>
9     <group>pfe-iot,mqtt,suricata</group>
10  </rule>
11
12  <!-- MQTT flood (DoS) -->
13  <rule id="100101" level="10">
14    <if_sid>100100</if_sid>
15    <field name="alert.signature_id">1000002</field>
16    <description>PFE -- MQTT DoS flood detecte (>50
        msg/10s)</description>
17    <group>pfe-iot,mqtt,dos</group>
18    <options>alert_by_email</options>
19  </rule>
20
21  <!-- Subscribe wildcard -->
22  <rule id="100102" level="8">
23    <if_sid>100100</if_sid>
24    <field name="alert.signature_id">1000003</field>
25    <description>PFE -- MQTT subscribe wildcard #
        detecte</description>
26    <group>pfe-iot,mqtt,policy-violation</group>
27  </rule>

```

```

28
29 <!-- Connexion MQTT sans TLS -->
30 <rule id="100103" level="12">
31   <if_sid>100100</if_sid>
32   <field name="alert.signature_id">1000001</field>
33   <description>PFE -- MQTT sans TLS (violation de
34     politique)</description>
35   <group>pfe-iot,mqtt,tls-violation</group>
36   <options>alert_by_email</options>
37 </rule>
38
39 <!-- Correlation : 3+ alertes MQTT du meme device en 60s -->
40 <rule id="100110" level="13" frequency="3" timeframe="60">
41   <if_matched_sid>100100</if_matched_sid>
42   <same_source_ip />
43   <description>PFE -- Activite suspecte repetee depuis le meme
44     device IoT</description>
45   <group>pfe-iot,mqtt,correlation</group>
46 </rule>
</group>

```

Listing B.5: local_rules.xml — Règles Wazuh pour corrélation MQTT/Suricata

B.6 Dockerfile du simulateur de flotte

```

1 FROM python:3.11-alpine
2
3 RUN apk add --no-cache ca-certificates openssl \
4   && update-ca-certificates
5
6 WORKDIR /app
7 COPY app/requirements.txt /app/requirements.txt
8 RUN pip install --no-cache-dir -r /app/requirements.txt
9
10 COPY app/fleet_sim.py /app/fleet_sim.py
11 COPY certs/ /app/certs/
12 COPY dataset/ /app/dataset/
13
14 CMD ["python", "/app/fleet_sim.py", \

```

```
15     "--csv",  
        "/app/dataset/iot_communication_security_dataset_sample.csv"]
```

Listing B.6: Dockerfile — Image Docker du simulateur de flotte IoT

B.7 Dockerfile Toolbox

```
1 FROM alpine:3.20  
2  
3 RUN apk add --no-cache \  
4     bash ca-certificates curl openssl \  
5     bind-tools iproute2 iputils tcpdump \  
6     net-tools nmap jq \  
7     python3 py3-pip git \  
8     busybox-extras tzdata \  
9     && update-ca-certificates  
10  
11 RUN adduser -D pfe && mkdir -p /work && chown -R pfe:pfe /work  
12 WORKDIR /work  
13 USER pfe  
14  
15 CMD ["bash"]
```

Listing B.7: Dockerfile — Image toolbox pour tests réseau et sécurité

B.8 Commandes de création des réseaux Docker

```
1 # VLAN 10 -- Zone IoT  
2 docker network create --driver bridge \  
3     --subnet 192.168.10.0/24 --gateway 192.168.10.1 \  
4     --opt com.docker.network.bridge.name=br-iot pfe-iot-network  
5  
6 # VLAN 20 -- Zone DMZ  
7 docker network create --driver bridge \  
8     --subnet 192.168.20.0/24 --gateway 192.168.20.1 \  
9     --opt com.docker.network.bridge.name=br-dmz pfe-dmz-network  
10  
11 # VLAN 30 -- Zone PKI  
12 docker network create --driver bridge \  
13     --subnet 192.168.30.0/24 --gateway 192.168.30.1 \  
14     --opt com.docker.network.bridge.name=br-pki pfe-pki-network
```



```

15
16 # VLAN 40 -- Zone SIEM
17 docker network create --driver bridge \
18   --subnet 192.168.40.0/24 --gateway 192.168.40.1 \
19   --opt com.docker.network.bridge.name=br-siem pfe-siem-network
20
21 # VLAN 50 -- Zone Analyse
22 docker network create --driver bridge \
23   --subnet 192.168.50.0/24 --gateway 192.168.50.1 \
24   --opt com.docker.network.bridge.name=br-analyse
    pfe-analyse-network

```

Listing B.8: Création des réseaux Docker par zone de sécurité

B.9 Variables d'environnement du projet

Tab. B.1 : Variables d'environnement principales du projet

Variable	Valeur	Description
VAULT_ADDR	http://192.168.30.10:8200	Adresse de l'API Vault
VAULT_TOKEN	<root_token>	Token d'authentification
MQTT_BROKER	192.168.20.10	IP du broker Mosquitto
MQTT_PORT_TLS	8883	Port MQTT sécurisé
MQTT_PORT_TCP	1883	Port MQTT non sécurisé
CA_CERT	/certs/ca-chain.crt	Chaîne CA (root + intermediate)
WAZUH_MANAGER	192.168.40.10	IP du manager Wazuh
SURICATA_EVE	/var/log/suricata/eve.json	Fichier événements Suricata

Résumé

Mots-clés : IoT, sécurité, TLS 1.3, mTLS, PKI, HashiCorp Vault, MQTT, Suricata, Wazuh, ML, GNS3.

Ce projet, mené avec *Tunisie Telecom*, conçoit et valide une architecture de sécurisation de bout en bout d'un écosystème IoT simulé sous GNS3. La solution associe une PKI HashiCorp Vault à deux niveaux, un broker MQTT en TLS 1.3/mTLS obligatoire, un IDS Suricata et un SIEM Wazuh. Un pipeline ML (IsolationForest + RandomForest) détecte les anomalies : $F1 = 0,994$ et $ROC-AUC = 0,999$ pour RF ; surcoût mTLS de l'ordre de +8 % sur le débit applicatif et +3,5 ms de latence, sans impact opérationnel notable.

Abstract

Keywords : IoT, security, TLS 1.3, mTLS, PKI, HashiCorp Vault, MQTT, Suricata, Wazuh, ML, GNS3.

This project, carried out with Tunisie Telecom, designs and validates an end-to-end security architecture for a simulated IoT ecosystem under GNS3. The solution combines a two-level HashiCorp Vault PKI, a Mosquitto MQTT broker enforcing TLS 1.3/mTLS, a Suricata IDS and a Wazuh SIEM. An ML pipeline (IsolationForest + RandomForest) performs anomaly detection : $F1 = 0.994$, $ROC-AUC = 0.999$ (RF) ; mTLS overhead is about +8 % on application throughput and +3.5 ms latency, with no noticeable operational impact.

ملخص

الكلمات المفتاحية: إنترنت الأشياء، أمن سيبراني، 1.3 TLS، Vault، HashiCorp PKI، mTLS، MQTT، Suricata، Wazuh تعلم آلي، GNS3.

صمم هذا المشروع، المنجز مع اتصالات تونس، بنية لتأمين الاتصالات من طرف إلى طرف داخل منظومة إنترنت الأشياء المحاكاة عبر GNS3. تجمع الحل بين بنية مفاتيح عمومية (HashiCorp Vault) ووسيط MQTT بـ TLS 1.3/mTLS، وIDS Suricata ومنصة SIEM. Wazuh يُحقق نموذج RandomForest في سلسلة التعلم الآلي $F1 = 0,994$ و $ROC-AUC = 0,999$ ؛ ولا تتجاوز كلفة mTLS +8 % على الإنتاجية و3,5 ميلي ثانية على زمن الاستجابة.